

run_stage1.py 运行结果详细解析

会话 ID: 019f8db3-bffc-7ff0-af5d-04a1d37d1d79 开始时间: 2026-07-23 08:40:58 消息数: 15

用户 2026-07-23 08:40:58

Files mentioned by the user:

[INFO] 随机种子已设置: 130 [INFO] device=cuda

=====...:

C:\Users\XiaoyanXiong\codex/attachments/12ed92ce-fd2b-4d0e-a547-8682f5666673/pasted-text.txt

My request for Codex:

!python /content/drive/MyDrive/s1/run_stage1.py \

--packet_csv /content/drive/MyDrive/s1/data/ar002_et12_20260511_001-stage1_packets.csv \

--flow_csv /content/drive/MyDrive/s1/data/ar002_et12_20260511_001-stage1_flows.csv \

--external_packet_csv /content/drive/MyDrive/s1/data/pbx23_20260512_001-stage1_packets.csv \

--external_flow_csv /content/drive/MyDrive/s1/data/pbx23_20260512_001-stage1_flows.csv \

--config

/content/drive/MyDrive/s1/0722/0722C_ar002_et12_20260511_001_external_csv/stage1_config_C.yaml \

--out_dir /content/drive/MyDrive/s1/0722/0722C_ar002_et12_20260511_001_external_csv \

--export_embeddings

运行结果。详细解析

助手 2026-07-23 08:41:03

我会先读取你附上的完整运行日志，并结合`run\_stage1.py`、配置文件及相关评估代码核对各项指标含义，然后按“数据→训练→阈值→内部/外部测试→风险与改进”解析。

助手 2026-07-23 08:41:14

日志已经揭示出一个很明确的结论：模型在同源时间后移验证集上表现很好，但在外部`pbx23`上发生了严重的分布迁移，且不仅是阈值失配——排序能力本身也反转/失效。下面我再提取完整训练轨迹和配置参数，区分哪些现象是正常过拟合、哪些是数据或语义管线问题。

助手 2026-07-23 08:43:11

总结结论

程序完整运行成功，模型、图表、预计算张量和三份 embedding 均已正常保存。

但模型的实际结论是：

> 在与训练集同源、时间相邻的验证集上效果很好；在外部`pbx23`数据上彻底失效。目前不能认为模型具备跨设备或跨场景泛化能力。

这不是简单地“阈值选错”，因为外部测试集的 ROC-AUC 只有 ****0.3481****，PR-AUC 只有 ****0.0400****，说明模型在外部域中连正负样本的排序关系都没有学对。

1. 数据集与切分

训练与验证数据

训练和验证均来自：

ar002_et12_20260511_001

采用 chronological 时间顺序切分：

数据集	流数量	正样本	负样本	正样本比例
Train	369,161	14,623	354,538	3.961%
Validation	52,737	2,021	50,716	3.832%
External test	1,263	71	1,192	5.622%

训练、验证的正样本比例非常接近，且来自同一个抓包文件。

更重要的是，日志中的时间范围显示：

- 训练集覆盖大约 **74.3 秒**
- 验证集覆盖紧接着的约 **10.6 秒**

所以验证集实际上是同一个网络环境、同一段捕获过程的相邻时间片。它能检验短时间内的时间外推，但不能代表跨设备泛化。

外部测试集来自：

pbx23_20260512_001

它和训练域的设备、时间、流量结构可能都不同。

每条流的平均报文数

由日志中的 packet 数量粗略计算：

数据集	packet 数量	flow 数量	平均 packets/flow
Train	2,939,475	369,161	约 7.96
Validation	292,982	52,737	约 5.56
External test	18,775	1,263	约 14.87

外部集的平均流长度明显更高。这已经是一个直观的域差异信号。

2. 输入特征与模型结构

方案 C

当前使用的是“分层特征注入”方案 C：

- packet 特征：35 维
- flow 特征：60 维
- 最大序列长度：128
- 截断策略：`head`
- Transformer 隐藏维度：128
- 注意力头：4
- Transformer 层数：2
- FFN 维度：256
- dropout：0.15
- 时间编码：启用
- 位置编码：启用

- flow 融合: `token\_film`
- 参数量: 401,252
- 模型大小: 约 1.53 MB

这属于规模较小的 Transformer，模型容量本身并不过大。

数据预处理

日志显示:

- 数值特征多数使用 `log1p`
- 使用 RobustScaler
- 使用训练分布的 0.1%–99.9% 分位数裁剪
- packet 数值、类别和二值特征拼接为 35 维
- flow 特征最终为 60 维

整体预处理设计是合理的，但外部数据必须严格确认:

- 使用的是训练集拟合的 scaler 和裁剪边界;
- packet/flow 列顺序完全一致;
- 单位完全一致;
- 缺失值处理一致;
- TCP 方向及 forward/backward 定义一致;
- 外部文件的 label 语义一致。

这些是目前最值得优先排查的地方。

3. 类别不平衡处理

原始训练集正样本比例为 3.96%，训练 sampler 将采样后的正样本比例控制为:

```
positive fraction = 0.1
```

同时使用 focal loss。

这意味着训练批次中的正样本比例高于真实分布，有利于模型学习少数类，但会产生两个影响:

1. 模型输出概率不一定是校准后的真实概率;
2. 默认 0.5 阈值不一定合适，需要在验证集上选阈值。

程序确实做了阈值搜索，所以这部分处理原则上正确。但采样和 focal loss 主要影响概率校准，无法解释外部 AUC 低于 0.5——AUC 失效表明问题比阈值更深。

4. 训练过程分析

最优模型

训练最大设置为 500 epoch，但在第 70 epoch 触发 early stopping。

验证 PR-AUC 的最佳值为:

```
PR-AUC = 0.8914278704
```

从保存日志看，最佳 checkpoint 出现在大约第 45 epoch 后。

训练趋势:

Epoch	Train loss	Val loss	Val class-1 F1	Val AUC
5	0.0535	0.0298	0.7671	0.9867
20	0.0438	0.0272	0.7879	0.9895
35	0.0386	0.0287	0.7863	0.9895
45	0.0351	0.0285	0.7972	0.9891
55	0.0330	0.0297	0.8025	0.9883
65	0.0322	0.0290	0.8062	0.9885
70	0.0316	0.0296	0.8022	0.9885

训练 loss 一直下降，而验证 loss 在约 20–30 epoch 后基本停止改善，说明后期存在轻微过拟合。不过验证 AUC 和 F1 没有崩溃，early stopping 行为正常。

因此，外部失败不能主要归因于“训练到 70 epoch 过拟合”；即使采用最佳 checkpoint，外部排序能力仍然非常差。

5. TokenFiLM 状态解析

训练初期，flow 特征对 packet token 的有效残差比例逐渐增大：

```
Epoch 1: residual_ratio ≈ 0.030
Epoch 20: residual_ratio ≈ 0.140
Epoch 22 : residual_ratio ≈ 0.150
```

从第 22 epoch 左右开始，残差比例一直被限制在 0.15。这说明 TokenFiLM 的原始调制强度继续增长，但最终有效注入量触发了上限裁剪。

例如后期：

```
raw_residual_ratio ≈ 0.27-0.29
residual_ratio      = 0.15
```

这不是数值爆炸，因为有效残差被控制住了。但它说明：

- 模型非常希望依赖 flow 特征；
- flow 分支的原始调制越来越强；
- 最终被 15% 的限制持续截断。

如果 flow 特征包含强烈的设备/环境特征，那么模型很容易在同源验证集上表现优秀，却在 `pbx23` 上失效。因此建议后续重点做：

- packet-only；
- flow-only；
- packet + flow；
- 去掉 TokenFiLM，仅 concat；
- 不同 `residual\_ratio` 上限；

这些消融实验可以判断失败是否主要来自 flow 域偏移。

6. 验证集表现

验证集的最优阈值约为：

```
threshold = 0.636734
```

对应混淆矩阵信息：

```
| 指标 | 数值 |
|---|---|
| TP | 1,635 |
| FP | 362 |
| TN | 50,354 |
| FN | 386 |
| Precision class 1 | 0.8187 |
| Recall class 1 | 0.8090 |
| F1 class 1 | 0.8138 |
| ROC-AUC | 0.9893 |
| PR-AUC | 0.8914 |
```

这是一个很好的同源验证结果。

但日志同时指出，不存在同时满足：

```
Precision ≥ 0.85
Recall ≥ 0.85
```

的阈值。当前模型在这个验证集上的实际能力上限大约是 class-1 F1 0.814，而不是接近完美。

阈值输出略有不一致

日志中依次出现：

```
0.629
0.64
0.637
0.636734
```

这大概率来自不同粒度的阈值搜索：

- 0.629：固定步长网格；
- 0.64：格式化或另一套搜索；
- 0.636734：精确候选阈值；
- 0.637：三位小数打印。

数值非常接近，不影响本次外部测试结论。不过为了实验可复现性，建议最终只保留一个阈值搜索函数和一个精确阈值来源。

7. 外部测试集结果

在验证阈值 `0.636734` 下：

```
| 指标 | 数值 |
|---|---|
| TP | 0 |
| FP | 21 |
| TN | 1,171 |
| FN | 71 |
| Precision class 1 | 0 |
| Recall class 1 | 0 |
| F1 class 1 | 0 |
| Accuracy | 92.72% |
| Macro-F1 | 0.4811 |
```

Weighted-F1	0.9081
ROC-AUC	0.3481
PR-AUC	0.0400

混淆矩阵为：

1171	21
71	0

也就是说：

- 71 个真实异常全部漏报；
- 21 个正常流被误报为异常；
- 模型没有正确识别任何一个异常流。

为什么 92.72% accuracy 是误导性的

测试集有 $1,192/1,263 = 94.38\%$ 的正常样本。

如果模型把所有样本都预测成正常，其 accuracy 会是：

$1192 / 1263 = 94.38\%$

当前模型 accuracy 只有 92.72%，甚至低于“全部预测正常”的简单基线。

因此这里不能使用 accuracy 或 weighted-F1 判断性能。真正重要的是：

- class-1 recall = 0;
- class-1 F1 = 0;
- macro-F1 ≈ 0.48 ;
- PR-AUC = 0.04。

对于入侵检测任务，这个外部结果不可用。

8. 为什么不是简单的阈值问题

测试集“oracle”搜索得到的最佳诊断阈值为：

threshold = 0.013636

结果却是：

TP = 71
FP = 1178
TN = 14
FN = 0
Precision = 0.05685
Recall = 1.0
F1 = 0.10758

它几乎把所有样本都预测成异常：

$= 71 + 1178 = 1249$
 $= 1263$

虽然召回率达到 1，但 1,192 个正常样本中有 1,178 个被误报，完全没有实用价值。

外部正样本比例本身为：

$71 / 1263 = 5.62\%$

PR-AUC 只有 4.00%，甚至低于约 5.62% 的随机排序基准。

ROC-AUC 为 0.348，也明显低于随机水平 0.5。这意味着外部域中，真实异常的预测分数总体上反而比正常流更低。

即使简单把分数反过来，AUC 也只有：

$$1 - 0.3481 = 0.6519$$

所以“标签完全反了”值得排查，但即使反转，也只能得到一般的区分能力，无法完全解释问题。

9. 最可能的原因

按优先级排序，我认为应检查以下问题。

第一优先级：外部标签语义或数据关联错误

重点确认：

- 两个文件中 `label=1` 是否都表示同一种异常；
- 外部 flow label 是否正确关联到 packet；
- `flow\_id` 是否发生错配、截断或类型转换；
- packet 与 flow 的 join 是否出现一对多、漏匹配或跨流匹配；
- 外部数据是否把 benign/attack 编码方向弄反；
- 71 个外部正样本是否真的是模型应该检测的攻击类型。

AUC 低于 0.5 和“所有正样本均低分”使标签/关联问题成为第一排查对象。

第二优先级：特征定义或单位不一致

尤其检查：

- 时间单位是秒、毫秒还是微秒；
- `flow\_duration` 单位是否一致；
- bytes/s、packets/s 是否采用相同公式；
- TCP forward/backward 方向定义是否一致；
- `init\_win\_bytes\_forward/backward` 的缺失值编码；
- 外部 categorical 值是否被映射到错误类别；
- CSV 列名相同但生成逻辑是否相同。

第三优先级：真实的设备域偏移

`ar002\_et12` 与 `pbx23` 很可能具有不同的：

- 协议组合；
- 服务端口结构；
- 包长度分布；
- 流持续时间；
- 上下行比例；
- TCP window；
- packet 数量；
- 正常通信模式；
- 攻击类型。

方案 C 有 60 个 flow 特征，并且 TokenFiLM 持续达到注入上限，模型可能学到了“ar002 环境特征”，而不是跨设备稳定的攻击特征。

第四优先级：验证设计过于同质

训练集和验证集来自同一个短时间捕获，且紧密相邻。高达 0.989 的验证 AUC 很可能高估了真实泛化性能。

目前验证集回答的是：

> 模型能否识别同一个设备、同一段流量中稍晚出现的类似模式？

它没有回答：

> 模型能否迁移到另一设备、另一天、另一网络环境？

10. 建议立即进行的诊断

在继续调 Transformer 参数之前，优先做以下实验。

1. 输出预测分数分布

分别统计 train、val、external 中：

label=0	score	P05/P25/P75/P95
label=1	score	P05/P25/P75/P95

并绘制四组分布：

- val negative;
- val positive;
- external negative;
- external positive。

这会直接显示外部正样本是否整体被压到低分区域。

2. 检查外部标签

抽取这 71 个正样本，输出：

- flow_id;
- source/destination;
- protocol;
- packet count;
- flow duration;
- 原始 label;
- 模型 score;
- 关联到的 packet 数量。

再人工核对若干条原始数据。

3. 比较特征分布

对 35 个 packet 特征和 60 个 flow 特征比较：

- 中位数/IQR;
- KS statistic;
- PSI;
- 缺失率;
- 零值率;
- 裁剪到上下限的比例;
- 未知类别比例。

如果外部大量值被裁剪到训练集边界，说明它已明显 OOD。

4. 做三个消融模型

至少运行：

```
packet-only
flow-only
packet + flow      C
```

如果 flow-only 在验证集很高、外部集极差，就基本可以确认 flow 特征产生了域捷径。

5. 测试外部数据自身是否可学习

将 `pbx23` 进行分层或时间切分，用其中一部分训练、一部分验证。

结果解释：

- 如果 pbx23 内部也学不好：标签质量、样本量或特征表达存在问题；
- 如果 pbx23 内部能学好，而 ar002→pbx23 失败：明确是域迁移问题；
- 如果混合两个域后性能明显恢复：需要多域训练或域适配。

6. 修复排序后再校准阈值

现在不建议仅通过降低阈值解决问题。因为外部 oracle 阈值已经证明：降低阈值只会把几乎所有正常流都报成异常。

必须先让外部：

```
ROC-AUC > 0.5
PR-AUC
```

之后才值得使用外部校准集选择阈值。

11. Embedding 导出结果

`--export\_embeddings` 已成功执行：

```
| 文件 | shape |
|---|---|
| `stage1_train_embeddings.npz` | `(369161, 128)` |
| `stage1_val_embeddings.npz` | `(52737, 128)` |
| `stage1_test_embeddings.npz` | `(1263, 128)` |
```

每份包含：

```
flow_id
label
z
split
flow_start_timestamp_us
source_id
destination_id
```

同时成功生成：

```
stage1_flow_metadata.csv
```

这些 embedding 可以继续用于 Stage 2，但需要注意：

> Stage 1 embedding 在外部域上已经表现出严重失配。Stage 2 即使在同源数据上表现良好，也可能继续继承这种域偏差。

建议先对三份 embedding 做 UMAP/PCA 可视化，分别用以下方式着色：

- train/val/test 域；
- label；
- source_id；
- destination_id。

如果 embedding 首先按数据来源分群、而不是按 label 分群，就能直观看出模型主要编码了设备域信息。

最终判断：本次训练过程正常、同源验证性能优秀、导出成功；核心问题是严重的跨域泛化失败，应优先检查外部标签/关联与特征一致性，其次才是模型结构和超参数。

用户 2026-07-23 10:33:00

Files mentioned by the user:

[INFO] 随机种子已设置: 130 [INFO] device=cuda

=====....:

C:\Users\XiaoyanXiong\.codex/attachments/1206197e-d4a4-4b12-897e-d84cd8cbf1a4/pasted-text.txt

My request for Codex:

方案B跑的结果。详细解析

助手 2026-07-23 10:33:06

我会把方案 B 的完整日志提取出来，并与刚才方案 C

在数据切分、最佳验证指标、阈值迁移和外部测试表现上逐项对照，重点判断 B 是否真正改善跨域泛化。

助手 2026-07-23 10:35:02

方案 B 运行结果解析

核心结论

方案 B 完整运行成功，但外部测试仍然失败。

与方案 C 相比：

- 方案 B 的同源验证性能略低；
- 外部 ROC-AUC 从 C 的 **0.3481** 提升到 B 的 **0.4469**；
- 外部 PR-AUC 从 **0.0400** 提升到 **0.0474**；
- 在验证阈值下，方案 B 检出了 2/71 个异常，而方案 C 是 0/71；
- 但方案 B 的外部 ROC-AUC 仍低于随机水平 0.5，PR-AUC 仍低于外部正样本率 5.62%。

因此，更准确的判断是：

> 去掉 flow 特征后，方案 B 的跨域排序能力比方案 C 稍好，说明方案 C 的 flow 特征/TokenFiLM 很可能加剧了域偏移；但 packet 特征本身依然存在严重跨域失效，方案 B 仍不可用于外部检测。

1. 数据切分

方案 B 和方案 C 使用了完全相同的数据：

数据集	来源	样本数	正样本	负样本	正样本率
---	---	---	---	---	---

```
| Train | ar002_et12 | 369,161 | 14,623 | 354,538 | 3.961% |  
| Validation | ar002_et12 | 52,737 | 2,021 | 50,716 | 3.832% |  
| External test | pbx23 | 1,263 | 71 | 1,192 | 5.622% |
```

训练和验证来自同一个 `ar002\_et12` 抓包，只是按时间前后切分；外部测试来自 `pbx23`。

所以：

- Validation 衡量同设备、相邻时间段泛化；
- External test 衡量跨设备/跨环境泛化。

这两个任务的难度完全不同。

2. 方案 B 的输入和模型

日志明确显示：

```
B - Packet  
Input dim: 35  
fusion_method=none  
use_flow_fusion=False
```

每条流被整理成：

```
X shape = (flow , 128, 35)
```

其中：

- 最大 packet 序列长度：128；
- packet 特征维度：35；
- 截断方式：`head`；
- 位置编码：启用；
- 时间编码：启用；
- flow 特征融合：禁用；
- TokenFiLM：不使用。

模型参数：

```
344,963
```

方案 C 参数量是 401,252，因此方案 B 少了：

```
401,252 - 344,963 = 56,289
```

约减少 14%。

方案 B 的主要配置：

```
| 参数 | 数值 |  
|---|---:|  
| d_model | 128 |  
| attention heads | 4 |  
| Transformer layers | 2 |  
| FFN dimension | 256 |  
| dropout | 0.20 |  
| max_seq_len | 128 |  
| input dimension | 35 |
```

```
| flow fusion | 否 |  
| loss | Focal loss |
```

3. 一个日志上的小问题

Preprocessor Summary 中显示:

```
flow_feature_dim: 68  
has_flow_features: True
```

但 Dataset 和模型明确显示:

```
No separate flow features  
use_flow_fusion=False  
fusion_method=None
```

因此本次模型实际没有使用这些 flow 特征。

这更像是预处理器对象仍然记录了“能够生成哪些 flow 特征”，而不是“本模型最终使用了哪些特征”。建议以后把日志字段拆开:

```
preprocessor_can_generate_flow_features  
dataset_contains_flow_features  
model_uses_flow_features
```

否则容易误以为方案 B 实际仍输入了 flow 特征。

另外，方案 B 的日志中 flow 特征定义是 68 维，而上一轮方案 C 是 60 维，说明两次配置或代码版本不完全相同。虽然 B 没有使用 flow 特征，但这仍提示两次实验不是严格完全相同的代码环境。

4. 类别不平衡处理

方案 B 使用:

```
sampler mode: inverse-frequency balanced  
loss: focal
```

这和上一轮方案 C 不同。

方案 C 使用的是:

```
controlled positive fraction = 0.1
```

方案 B 的 inverse-frequency balanced 通常意味着 sampler 将训练批次近似平衡到:

```
label 0 ≈ 50%  
label 1 ≈ 50%
```

但原始数据的正样本率只有 3.96%。

因此，方案 B 同时使用了:

- 强平衡采样;
- focal loss。

这可能产生“少数类双重补偿”。它有利于模型关注异常，但也会:

- 改变预测概率的校准;
- 增加假阳性;

- 使最佳阈值偏离 0.5;
- 让不同实验之间难以只归因于模型结构。

事实上, B 的验证最优阈值约为 0.482, 而 C 是 0.637, 二者概率尺度明显不同。

5. 训练过程

训练最大设置为 500 epoch, 但在第 60 epoch early stopping。

主要训练轨迹:

Epoch	Train loss	Val loss	Val class-1 F1	Val AUC
5	0.03743	0.03321	0.6917	0.9808
10	0.03273	0.02860	0.7416	0.9850
15	0.03105	0.02728	0.7624	0.9873
20	0.03006	0.02627	0.7697	0.9884
25	0.02916	0.02570	0.7786	0.9883
30	0.02834	0.02548	0.7766	0.9887
35	0.02809	0.02546	0.7828	0.9886
40	0.02775	0.02509	0.7895	0.9889
45	0.02722	0.02511	0.7925	0.9891
50	0.02667	0.02548	0.7909	0.9890
55	0.02617	0.02527	0.7950	0.9892
60	0.02580	0.02599	0.7940	0.9890

训练过程相当稳定:

- train loss 持续下降;
- validation loss 在 40 epoch 左右达到平台;
- validation AUC 稳定在 0.989 左右;
- 后期没有明显崩溃;
- early stopping 正常执行。

最佳 checkpoint 的验证 PR-AUC 为:

0.8759686726

最佳模型大约出现在 45–50 epoch 区间, 而不是最后的 epoch 60。

6. 验证集结果

最佳验证阈值

网格搜索得到:

threshold $\approx$ 0.482

精确 oracle 阈值为:

threshold = 0.4826341271

验证集结果:

```
| 指标 | 数值 |
|---|---|
| TP | 1,608 |
| FP | 408 |
| TN | 50,308 |
| FN | 413 |
| Precision class 1 | 0.7976 |
| Recall class 1 | 0.7956 |
| F1 class 1 | 0.7966 |
| ROC-AUC | 0.9890 |
| PR-AUC | 0.8760 |
```

混淆矩阵:

50308	408
413	1608

验证集上仍然不存在同时满足:

```
Precision  $\geq$  0.85
Recall  $\geq$  0.85
```

的阈值。

方案 B 的实际同源上限约为:

```
class-1 F1  $\approx$  0.797
```

这是不错的结果，但比方案 C 略低。

7. 外部测试结果

使用验证集确定的阈值 `0.482634`:

```
| 指标 | 数值 |
|---|---|
| TP | 2 |
| FP | 68 |
| TN | 1,124 |
| FN | 69 |
| Precision class 1 | 2.86% |
| Recall class 1 | 2.82% |
| F1 class 1 | 2.84% |
| Accuracy | 89.15% |
| Macro-F1 | 0.4855 |
| Weighted-F1 | 0.8912 |
| ROC-AUC | 0.4469 |
| PR-AUC | 0.0474 |
```

混淆矩阵:

1124	68
69	2

含义非常直接:

- 71 个异常只识别出 2 个;
- 漏掉 69 个异常;
- 同时错误报警 68 个正常流;
- 预测出的 70 个异常中, 只有 2 个是真的。

对于入侵检测来说, 这个结果不可用。

8. 为什么 Accuracy 不能使用

测试集正常样本占:

$$1192 / 1263 = 94.38\%$$

如果直接把全部样本预测为正常:

$$\text{Accuracy} = 94.38\%$$

方案 B 的实际 Accuracy 是:

$$89.15\%$$

因此它比“全部预测正常”还差约 5.23 个百分点。

Weighted-F1 为 0.8912 也主要由大量正常样本贡献, 不能代表异常检测能力。

本任务应该重点观察:

- class-1 recall;
- class-1 precision;
- class-1 F1;
- macro-F1;
- PR-AUC;
- 每固定流量的误报数。

9. 测试集 Oracle 阈值说明

为了诊断, 程序在测试集标签上搜索了理论最佳阈值:

$$\text{threshold} = 0.121540$$

结果:

指标	数值
TP	62
FP	895
TN	297
FN	9
Precision	6.48%
Recall	87.32%
F1	0.1206

虽然 recall 达到 87.3%, 但代价是:

$$\frac{895}{1192 + 895} = 895 / 2087 = 75.08\%$$

预测为异常的流共有：

$$62 + 895 = 957$$

其中真正异常只有 62 条。

也就是说，每报出大约 15 个异常告警，只有 1 个是真的：

$$1 / 0.0648 \approx 15.4$$

所以不能通过简单降低阈值修复外部问题。

10. ROC-AUC 和 PR-AUC 的含义

ROC-AUC = 0.4469

ROC-AUC 小于 0.5 表示：

> 在外部数据上，随机抽取一个真实异常和一个真实正常，模型给异常更高分的概率只有约 44.7%。

它已经接近随机，但仍略低于随机。

如果简单将分数方向反转：

$$1 - 0.4469 = 0.5531$$

也只会得到很弱的区分能力。因此这里不是一个简单的 label 0/1 完全反转问题。

PR-AUC = 0.0474

外部测试集正样本率是：

$$71 / 1263 = 0.0562$$

随机排序的 PR-AUC 基线约等于正样本率，即 0.0562。

方案 B 的 PR-AUC：

$$0.0474 < 0.0562$$

仍低于随机基线。

所以尽管方案 B 比 C 有改善，它还没有达到“具备有效外部排序能力”的程度。

11. 方案 B 与方案 C 对比

指标	方案 B: packet-only	方案 C: packet + flow TokenFiLM	更好
--- ---: ---: ---			
参数量	344,963	401,252	B 更小
Dropout	0.20	0.15	不同配置
Sampler	inverse-frequency balanced	positive fraction=0.1	不同配置
Val PR-AUC	0.8760	0.8914	C
Val ROC-AUC	0.9890	0.9893	基本相同
Val class-1 F1	0.7966	0.8138	C
Val threshold	0.4826	0.6367	不可直接比较
Test ROC-AUC	0.4469	0.3481	B
Test PR-AUC	0.0474	0.0400	B
Test class-1 F1	0.0284	0	B, 但都不可用
Test TP	2	0	B


```
| Test FP | 68 | 21 | C |  
| Test FN | 69 | 71 | B |  
| Test oracle F1 | 0.1206 | 0.1076 | B |
```

对比解释

方案 C 在验证集上的优势：

```
PR-AUC: +0.0155  
class-1 F1: +0.0172
```

说明 flow 特征在同源数据中确实提供了额外预测信息。

但是在外部集上，方案 C 的 AUC 从 B 的 0.4469 降到 0.3481。这支持一个重要推断：

> flow 特征可能携带了大量与设备、网络环境或捕获条件相关的信息，使模型更容易拟合同源数据，但加重了跨设备域偏移。

不过，这个结论目前只能称为“有证据支持”，还不能称为严格的消融结论，因为两次实验还有其他配置差异：

- B dropout = 0.20, C dropout = 0.15;
- B 使用 inverse-frequency balanced sampler;
- C 使用 positive fraction=0.1 sampler;
- 两次 flow 特征定义分别显示为 68 和 60 维;
- 输出目录日期不同，可能对应不同配置或代码版本。

因此不能把 B/C 的全部差异都归因于“是否使用 flow 特征”。

12. B 的 sampler 可能带来的影响

方案 B 使用近似 50:50 的平衡采样，而 C 只将正样本比例提高到 10%。

这可能解释部分现象：

- B 的验证阈值更低;
- B 在外部集预测了更多正样本;
- B 的 FP 是 68，明显高于 C 的 21;
- B 能检出 2 个异常，但误报同时显著增加。

不过 sampler 主要影响概率尺度和分类边界，不能解释 AUC 小于 0.5。因为 ROC-AUC 与固定阈值无关。

所以 sampler 不是外部失败的根本原因，但它让 B/C 的阈值结果无法直接公平比较。

13. 目前能够得到的模型结论

结论一：flow 特征提高了同源性能

C 的验证 PR-AUC 和 class-1 F1 都高于 B，说明 flow 特征对于识别 `ar002\_et12` 内部异常模式有效。

结论二：flow 特征可能放大域依赖

C 的外部 AUC 显著低于 B，说明 flow 统计量可能包含：

- 设备行为;
- 服务类型;
- flow 长度;

- 上下行结构;
- TCP window;
- 连接持续时间;
- 捕获环境特征。

这些特征在训练域中非常有预测力，但在 `pbx23` 中关系发生变化。

结论三：packet-only 也没有解决域偏移

B 的外部 AUC 仍低于 0.5，说明问题不限于 flow 特征。packet 层仍可能存在：

- TTL 差异;
- packet length 分布差异;
- protocol 分布差异;
- TCP flags 行为差异;
- direction 定义差异;
- 时间间隔差异;
- 攻击类型差异;
- 标签或 flow-packet 关联问题。

结论四：同源验证集严重高估部署性能

B 和 C 的验证 AUC 都约为 0.989，但外部 AUC 分别只有 0.447 和 0.348。

这说明当前验证设计无法代表真实部署环境。

14. 下一步建议

第一优先级：做严格公平的 B/C 对照

两套实验必须统一：

```
seed: 130
dropout:
sampler:
loss:
batch_size:
learning_rate:
early_stopping:
preprocessing:
feature      :
```

唯一变化应当是：

```
B: packet-only
C: packet + flow
```

建议 sampler 都使用：

```
controlled_positive_fraction = 0.1
```

然后至少运行 3-5 个种子。否则单个 seed 的约 0.01-0.02 差异可能只是随机波动。

第二优先级：检查外部 score 分布

分别输出：

```
| Domain | Label | score mean/median/P05/P95 |
|---|---|---
```

```
| Validation | 0 | ... |
| Validation | 1 | ... |
| External | 0 | ... |
| External | 1 | ... |
```

当前 AUC 小于 0.5, 预计 external label-1 的分数中位数很可能低于 label-0。

第三优先级: 检查 packet 特征域偏移

由于方案 B 只用 packet 特征, 重点比较以下 35 维输入:

- `pkt\_len`
- `ip\_len`
- `payload\_len`
- `ttl\_or\_hop\_limit`
- `tcp\_window`
- `tcp\_header\_len`
- `udp\_len`
- `direction`
- `protocol`
- TCP flags
- ICMP type/code
- packet 时间间隔

尤其检查:

- TTL 是否直接暴露设备类型;
- direction 是否在两个数据源中含义一致;
- TCP window 是否有明显设备指纹;
- 外部异常是否使用了训练中没出现的协议;
- 外部数据是否存在大量裁剪饱和。

第四优先级: 确认外部标签和关联

对 71 个外部正样本逐条核验:

```
flow_id
label
packet count
source/destination
protocol
attack type
model score
```

还应检查:

- flow_id 在 packet CSV 中是否都能找到;
- packet 是否被关联到正确 flow;
- 是否存在 hash/整数精度问题;
- label=1 是否在两个数据源中语义一致;
- 外部攻击类型是否在训练数据出现过。

第五优先级: 增加域感知验证

更可靠的实验设计应当是:

/

如果目前只有两个域，可以尝试：

1. ar002 训练 → pbx23 测试；
2. pbx23 训练 → ar002 测试；
3. 两域混合训练，并按时间留出；
4. leave-one-device-out。

最终评价

方案 B 的训练过程和同源验证结果都正常，导出的三份 128 维 embedding 也成功生成。

从科学结论上看：

> 方案 B 比方案 C 更不依赖 flow 域特征，因此外部 AUC 从 0.348 提升到 0.447；但它仍没有超过随机水平，外部异常召回率仅为 2.82%。这说明去掉 flow 特征只是减轻了问题，并没有解决跨设备域偏移。

当前最值得做的不是继续调层数、注意力头或阈值，而是先完成：

1. 外部标签和 flow-packet 关联核查；
2. packet 特征分布与 score 分布比较；
3. 使用完全相同训练配置重跑严格的 B/C 消融；
4. 引入跨设备验证集。

只有当外部 ROC-AUC 稳定高于 0.5、PR-AUC 明显高于 5.62% 基线之后，阈值优化才真正有意义。

用户 2026-07-27 10:35:42

```
!python /content/drive/MyDrive/s1/run_stage1.py \
--packet_csv /content/drive/MyDrive/s1/data/ar002_et12_20260511_001-stage1_packets.csv \
--flow_csv /content/drive/MyDrive/s1/data/ar002_et12_20260511_001-stage1_flows.csv \
--external_packet_csv
/content/drive/MyDrive/s1/data/Wednesday-workingHours-external5000-stage1_packets.csv \
--external_flow_csv
/content/drive/MyDrive/s1/data/Wednesday-workingHours-external5000-stage1_flows.csv \
--config /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/stage1_config_B_
Both.yaml \
--out_dir /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/B_external_Wed
nesday_workingHour \
--export_embeddings
[TEST] Full metrics: {'accuracy': 0.6528, 'macro_precision': 0.7867011892336879, 'macro_recall':
0.6077991924597564, 'macro_f1': 0.5620546378212037, 'weighted_precision': 0.7669164951109089,
'weighted_recall': 0.6528, 'weighted_f1': 0.5854187776440396, 'precision_label1': 0.9555125725338491,
'recall_label1': 0.22383325781603988, 'f1_label1': 0.36270190895741555, 'auc': 0.7707969840453293,
'pr_auc': 0.7078082564015336, 'per_class_f1': {'0': 0.7614073666849918, '1': 0.36270190895741555},
'confusion_matrix': [[2770, 23], [1713, 494]], 'num_classes': 2, 'num_samples': 5000, 'loss':
0.6717914450913668, 'threshold': 0.48200000000000003}
```

详细解析这个结果。

助手 2026-07-27 10:37:11

核心结论

这次方案 B 在 `Wednesday-workingHours-external5000` 上的表现，明显优于此前在 `pbx23` 外部集上的结果：

- ROC-AUC: **0.7708**
- PR-AUC: **0.7078**
- 异常精确率: **95.55%**
- 异常召回率: **22.38%**
- 异常 F1: **0.3627**

说明模型已经具备有效的跨数据集排序能力，不再是接近随机或反向排序。

但是，使用从 `ar002` 验证集得到的阈值 `0.482` 后，模型非常保守：

> 报出的异常几乎都是真的，但漏掉了约 77.6% 的真实异常。

因此，这次的主要问题已经从“模型完全无法跨域区分”变成了：

> 模型能区分一部分外部异常，但原验证阈值无法直接迁移到 Wednesday 数据，导致召回率过低。

1. 外部测试集组成

由混淆矩阵：

```
[[2770, 23],  
 [1713, 494]]
```

可以还原数据分布：

类别	数量	比例
正常 label 0	2,793	55.86%
异常 label 1	2,207	44.14%
总计	5,000	100%

这个外部集并不是高度类别不平衡数据，异常比例高达 44.14%。

这和训练数据差异很大：

数据集	异常比例
ar002 Train	3.96%
ar002 Validation	3.83%
Wednesday external5000	44.14%

因此存在明显的 label-prior shift，即类别先验发生变化。

但仅仅类别比例改变不会直接降低 ROC-AUC；它主要会影响：

- 最合适的决策阈值；
- Precision；
- Accuracy；
- 概率校准；
- 实际预测出的异常比例。

2. 混淆矩阵逐项解释

2770	23
1713	494

分别表示：

- TN = 2,770：正常流被正确识别；
- FP = 23：正常流被错误报警；
- FN = 1,713：异常流被漏检；
- TP = 494：异常流被成功检出。

模型一共预测了：

$$\begin{aligned} &= 494 + 23 = 517 \\ &= 2770 + 1713 = 4483 \end{aligned}$$

预测异常比例只有：

$$517 / 5000 = 10.34\%$$

但真实异常比例是：

$$2207 / 5000 = 44.14\%$$

模型对外部异常数量存在严重低估，预测异常比例只有真实异常比例的大约四分之一。

3. 异常类指标

Precision = 95.55%

计算为：

$$494 / (494 + 23) = 95.55\%$$

含义是：

> 模型发出的 517 个异常告警中，494 个是真的，只有 23 个是误报。

这是非常高的告警可信度。

平均来看，大约每：

$$1 / (1 - 0.9555) \approx 22.5$$

个异常告警中才有一个误报。

如果模型定位为“高置信度告警筛选器”，这个结果很有价值。

Recall = 22.38%

计算为：

$$494 / (494 + 1713) = 22.38\%$$

含义是：

> 2,207 个真实异常中，模型只检测出 494 个，漏掉了 1,713 个。

漏报率为：

$$1713 / 2207 = 77.62\%$$

对于完整的入侵检测系统，这个召回率太低。模型目前只能发现其中最明显、最接近训练分布的异常。

F1 = 0.3627

F1 是 Precision 和 Recall 的调和平均：

$$\begin{aligned} & 2 \times 0.9555 \times 0.2238 / (0.9555 + 0.2238) \\ & \approx 0.3627 \end{aligned}$$

虽然 Precision 极高，但 Recall 太低，所以 F1 只有 0.363。

4. 正常类表现

Specificity

正常类召回率，也就是特异度：

$$\begin{aligned} & TN / (TN + FP) \\ & = 2770 / 2793 \\ & = 99.18\% \end{aligned}$$

说明正常流几乎都能正确通过，正常流误报率仅为：

$$23 / 2793 = 0.82\%$$

这个误报率相当低。

正常预测的可信度

预测为正常的 4,483 个样本中：

- 2,770 个确实正常；
- 1,713 个其实是异常。

所以正常预测的可信度为：

$$2770 / 4483 = 61.79\%$$

也就是说：

> 虽然异常告警很可信，但“预测正常”并不可靠；每 100 个被放行的流中，大约有 38 个实际上是异常。

这是当前模型最严重的部署风险。

5. Accuracy = 65.28% 如何理解

计算为：

$$(2770 + 494) / 5000 = 65.28\%$$

如果永远预测数量最多的正常类，基线 Accuracy 是：

$$2793 / 5000 = 55.86\%$$

当前模型比多数类基线提高：

$$65.28\% - 55.86\% = 9.42\%$$

所以这次 accuracy 确实包含一定有效信息，不像之前异常比例很低时那样完全具有欺骗性。

不过 accuracy 仍不足以描述当前问题，因为模型的错误严重偏向异常漏报：

- FP 只有 23;
- FN 却有 1,713。

在安全检测场景中，这两类错误的成本通常并不相同。

6. Macro Precision 为什么很高

日志给出：

```
macro_precision = 0.7867
macro_recall     = 0.6078
macro_f1         = 0.5621
```

Macro Precision

异常类 precision:

0.9555

正常类 precision:

$2770 / (2770 + 1713) = 0.6179$

平均为:

$(0.9555 + 0.6179) / 2 = 0.7867$

所以 Macro Precision 很高，主要因为异常告警非常准确。

Macro Recall

正常类 recall:

$2770 / 2793 = 0.9918$

异常类 recall:

$494 / 2207 = 0.2238$

平均为:

$(0.9918 + 0.2238) / 2 = 0.6078$

Macro Recall 也等于二分类中的 Balanced Accuracy。

它表明模型平均类别召回能力只有约 60.8%，原因是两类召回非常不均衡。

Macro F1

两类 F1:

类别	F1
正常类	0.7614
异常类	0.3627

平均:

$(0.7614 + 0.3627) / 2 = 0.5621$

因此 Macro-F1 比 Accuracy 更准确地揭示了少数性能严重不均衡的问题。

7. ROC-AUC = 0.7708

这是本次结果中非常重要的积极信号。

ROC-AUC 的直观含义是：

> 随机选择一个异常样本和一个正常样本，模型有约 77.08% 的概率给异常样本更高的异常分数。

与之前的 `pbx23` 结果对比：

```
| 外部集 | ROC-AUC |  
|---|---|  
| pbx23 | 0.4469 |  
| Wednesday | 0.7708 |
```

这说明模型在 Wednesday 数据上已经具有实质性的排序能力。

ROC-AUC 与当前阈值 0.482 无关。因此，即使当前 Recall 很低，也不能简单判断模型“没有学会”。更准确的解释是：

> 模型分数包含有效信息，但使用了一个非常保守、没有针对 Wednesday 校准的阈值。

不过 AUC 0.771 仍属于中等水平，不是非常强。正负分数分布仍然存在较大重叠。

8. PR-AUC = 0.7078

Wednesday 外部集的异常比例为：

```
2207 / 5000 = 0.4414
```

随机模型的 PR-AUC 基线约等于异常比例，即：

```
0.4414
```

实际 PR-AUC 是：

```
0.7078
```

明显高于随机基线，绝对提升约：

```
0.7078 - 0.4414 = 0.2664
```

这进一步说明模型不是随机模型。

与此前 `pbx23` 对比：

```
| 外部集 | 正样本率 | PR-AUC | 是否超过随机基线 |  
|---|---|---|---|  
| pbx23 | 5.62% | 4.74% | 否 |  
| Wednesday | 44.14% | 70.78% | 是，明显超过 |
```

因此，模型对不同外部域的泛化差别极大：

- 对 `pbx23` 几乎失效；
- 对 Wednesday 有明显区分能力。

这说明“是否来自外部域”不是唯一因素，外部攻击类型与训练攻击模式的相似程度也很重要。

9. 为什么阈值 0.482 在这里不合适

`0.482` 是从 `ar002` 验证集上根据 F1 得到的阈值。它对应的是：

- `ar002` 的特征分布;
- 约 3.83% 的异常率;
- `ar002` 的攻击类型;
- `ar002` 的模型分数校准状态。

Wednesday 的异常率达到 44.14%，并且特征分布不同。同一个数值阈值不一定还能表示相同的风险含义。

当前阈值产生的行为是：

```
Precision = 95.55%
Recall    = 22.38%
FPR       = 0.82%
```

这说明 0.482 实际上变成了一个“高置信度异常阈值”。

降低阈值大概率能够：

- 大幅提高 Recall;
- 降低 Precision;
- 增加 FP;
- 提高或降低 F1，取决于具体分数分布。

因为 PR-AUC 达到 0.708，所以存在获得比当前 0.363 更高 F1 的可能性。但没有完整预测分数，无法计算最佳阈值和理论 F1 上限。

10. Loss = 0.6718 为什么较高

测试 loss 为：

```
0.6718
```

它明显高于此前同源验证 loss。

这通常说明：

- 大量外部样本的概率预测与标签不一致;
- 许多真实异常被模型以较高置信度判为正常;
- 外部概率没有良好校准;
- 分类排序尚可，但概率数值不可直接解释为真实风险概率。

Loss 和 AUC 衡量的不是同一件事：

- AUC 关注正负样本排序;
- Loss 关注每个样本概率是否接近真实标签;
- Threshold metrics 关注某个固定决策点。

所以完全可能同时出现：

```
AUC    0.7708
Loss   0.6718
Recall 0.2238
```

这正是“排序有效、校准和阈值迁移较差”的典型表现。

11. 与此前方案 B 的 pbx23 结果比较

指标	pbx23	Wednesday	变化
---	---	---	---

测试样本数	1,263	5,000	—
异常率	5.62%	44.14%	差异很大
Accuracy	0.8915	0.6528	不可直接比较
Precision label 1	0.0286	0.9555	巨幅提升
Recall label 1	0.0282	0.2238	明显提升
F1 label 1	0.0284	0.3627	明显提升
ROC-AUC	0.4469	0.7708	巨幅提升
PR-AUC	0.0474	0.7078	巨幅提升
FP	68	23	更少
TP	2	494	更多

Accuracy 在 Wednesday 上反而更低，不代表模型变差，因为两个外部集类别比例完全不同。

真正具有比较价值的是：

- ROC-AUC;
- 相对于正样本基线的 PR-AUC;
- class-1 Precision/Recall/F1;
- FPR。

从这些指标看，Wednesday 的结果远好于 pbx23。

12. 这个结果适合什么用途

如果目标是高置信度告警

当前阈值很有吸引力：

- Precision 95.55%;
- FPR 0.82%;
- 517 个告警中只有 23 个误报。

可以把它看作第一层“高置信度异常筛选器”。

如果目标是完整检测攻击

当前模型不合格：

- Recall 只有 22.38%;
- 漏掉 1,713/2,207 个异常;
- 预测正常的样本中包含大量异常。

因此不能把低分流量直接视为安全。

比较合理的设计可能是：

Stage 2

13. 下一步最重要的实验

1. 绘制 Wednesday 分数分布

分别绘制：

```
label=0
label=1
```

并标出阈值 0.482。

这可以直接显示为什么大量异常落在阈值以下。

2. 输出多阈值指标表

建议至少输出：

Threshold	Precision	Recall	F1	FPR	TP	FP
0.05						
0.10						
0.20						
0.30						
0.40						
0.482	0.9555	0.2238	0.3627	0.0082	494	23

并额外计算：

- 最大 F1 阈值；
- Recall ≥ 0.80 时的最佳 Precision；
- Recall ≥ 0.90 时的最佳 Precision；
- FPR $\leq 1\%$ 时的最佳 Recall；
- FPR $\leq 5\%$ 时的最佳 Recall。

这比只报告一个 F1 最优阈值更适合 NIDS。

3. 不要在最终测试集上直接调阈值

应该把 Wednesday 数据分成：

例如：

- 1,000 条用于阈值/概率校准；
- 4,000 条作为最终测试；

更好的是按时间切分，避免相邻流泄漏。

阈值只能在校准集确定，然后锁定到最终测试集。

4. 按攻击类型拆分 Recall

Wednesday 数据通常可能包含多种攻击或多个时间阶段。应报告：

attack type	
attack type	TP/FN
attack type	Recall
attack type	score

当前 494 个被识别的异常可能集中于某一种攻击，而其他攻击类型完全漏检。只看整体 Recall 无法判断这一点。

5. 检查 external5000 的抽样方式

必须说明这 5,000 条是：

- 随机抽样;
- 分层抽样;
- head;
- 连续时间片;
- 人为平衡;
- 每个攻击类型固定抽样。

如果是人为构造或分层抽样，44.14% 的异常率可能不代表真实部署比例，因此：

- Precision 95.55% 不能直接外推到真实网络;
- Accuracy 也不能直接代表线上表现;
- ROC-AUC 相对更稳定;
- PR-AUC 仍受类别比例影响。

最终评价

这次结果有明显进步，而且性质上与 `pbx23` 完全不同：

> 模型在 Wednesday 外部集上已经学到了可迁移的异常排序信号，ROC-AUC 0.771、PR-AUC 0.708 都具有实际意义。

但当前阈值表现极端保守：

- 正常流误报率仅 0.82%;
- 告警精确率高达 95.55%;
- 异常召回率却只有 22.38%。

所以当前模型适合作为“高置信度告警器”，还不适合作为完整入侵检测器。下一步重点不是马上重新训练，而是先分析预测分数和攻击类型，并使用独立的 Wednesday 校准集重新确定阈值。

用户 2026-07-27 10:38:12

```
!python /content/drive/MyDrive/s1/run_stage1.py \
--packet_csv /content/drive/MyDrive/s1/data/ar002_et12_20260511_001-stage1_packets.csv \
--flow_csv /content/drive/MyDrive/s1/data/ar002_et12_20260511_001-stage1_flows.csv \
--external_packet_csv
/content/drive/MyDrive/s1/data/Wednesday-workingHours-external5000-stage1_packets.csv \
--external_flow_csv
/content/drive/MyDrive/s1/data/Wednesday-workingHours-external5000-stage1_flows.csv \
--config [INFO] 随机种子已设置: 130
[INFO] device=cuda

=====
[STEP 1] Building dataloaders...
=====
[INFO] pipeline.py ----- build_dataloaders --- start
[INFO] pipeline.py ----- build_dataloaders --- split_method chronological
[INFO] data_io.py ----- read_stage1_csvs --- start
[data_io.py] --- read_stage1_csvs--- seed = 130, strategy = head, max_seq_len = 128
[INFO] data_io.py ----- read_stage1_csvs --- end
[INFO] data_io.py ----- read_stage1_csvs --- start
[data_io.py] --- read_stage1_csvs--- seed = 130, strategy = head, max_seq_len = 128
```

```

[INFO] data_io.py ----- read_stage1_csvs --- end

=====
[INFO] Split report: chronological_train_val_split_for_external_test
=====
[INFO] train:
num_flows = 369161
label_counts = {0: 354538, 1: 14623}
label1_ratio = 0.039611
time_range = [1778495176848505, 1778495251159359]
[INFO] val:
num_flows = 52737
label_counts = {0: 50716, 1: 2021}
label1_ratio = 0.038322
time_range = [1778495251159598, 1778495261763096]
=====

[INFO] splits.py ----- chronological_train_val_split_for_external_test
每个packet都包含flow信息的方案是 = B
[INFO] data_io.py ----- generate_and_save_stage1_tensors --- start
[data_io.py] packet_x_all shape: (2939475, 35)
[data_io.py] packet_x_all sample:
[[-0.02950557 -0.03237712 -0.03650611 0.5882353 -0.00169567 0.
 0. 1. 0. 0. 0. 0.
 0. 0. 1. 1. 0. 0.
 1. 0. 0. 0. 0. 0.
 1. 0. 0. 0. 0. 0.
 1. 1. 0. 0. 0. 0. ]]
[data_io.py] 模式: 方案B - 仅Packet特征
[INFO] saved precomputed stage1 tensors: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingH
ours_external_csv/B_external_Wednesday_workingHour/precomputed/seqLen128headtrainB.npz
[INFO] Keys in saved file: ['x', 'time', 'mask', 'labels', 'flow_ids', 'flow_start_timestamp_us', 'source_id',
'destination_id']
[INFO] data_io.py ----- generate_and_save_stage1_tensors_fast --- end
每个packet都包含flow信息的方案是 = B
[INFO] data_io.py ----- generate_and_save_stage1_tensors --- start
[data_io.py] packet_x_all shape: (292982, 35)
[data_io.py] packet_x_all sample:
[[-0.11866869 -0.13415103 -0.5701738 -0.3529412 0.79766387 0.19134793
 0. 1. 0. 0. 0. 0.
 0. 0. 1. 1. 0. 0.
 1. 0. 0. 0. 0. 0.
 1. 0. 0. 0. 1. 0.
 0. 0. 0. 0. 0. 0. ]]
[data_io.py] 模式: 方案B - 仅Packet特征

```

```

[INFO] saved precomputed stage1 tensors: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingH
ours_external_csv/B_external_Wednesday_workingHour/precomputed/seqLen128headvalB.npz
[INFO] Keys in saved file: ['x', 'time', 'mask', 'labels', 'flow_ids', 'flow_start_timestamp_us', 'source_id',
'destination_id']
[INFO] data_io.py ----- generate_and_save_stage1_tensors_fast --- end
每个packet都包含flow信息的方案是 = B
[INFO] data_io.py ----- generate_and_save_stage1_tensors --- start
[data_io.py] packet_x_all shape: (66618, 35)
[data_io.py] packet_x_all sample:
[[-0.13217773 -0.12897852 -0.07286663 0.60294116 -0.79766387 -0.8707324
 1.0430166 1. 0. 0. 1. 0.
 0. 0. 0. 1. 0. 0.
 1. 0. 0. 0. 0. 0.
 0. 1. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0.]]
[data_io.py] 模式: 方案B - 仅Packet特征
[INFO] saved precomputed stage1 tensors: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingH
ours_external_csv/B_external_Wednesday_workingHour/precomputed/seqLen128headtestB.npz
[INFO] Keys in saved file: ['x', 'time', 'mask', 'labels', 'flow_ids', 'flow_start_timestamp_us', 'source_id',
'destination_id']
[INFO] data_io.py ----- generate_and_save_stage1_tensors_fast --- end
[INFO] Dataset ----- Loading /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external
_csv/B_external_Wednesday_workingHour/precomputed/seqLen128headtrainB.npz
[INFO] Dataset ----- No separate flow features (方案A or 方案B)
[INFO] Dataset ----- Loaded 369161 flows
[INFO] X shape: (369161, 128, 35)
[INFO] Time shape: (369161, 128)
[INFO] Mask shape: (369161, 128)
[INFO] Dataset ----- Loading /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external
_csv/B_external_Wednesday_workingHour/precomputed/seqLen128headvalB.npz
[INFO] Dataset ----- No separate flow features (方案A or 方案B)
[INFO] Dataset ----- Loaded 52737 flows
[INFO] X shape: (52737, 128, 35)
[INFO] Time shape: (52737, 128)
[INFO] Mask shape: (52737, 128)
[INFO] Dataset ----- Loading /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external
_csv/B_external_Wednesday_workingHour/precomputed/seqLen128headtestB.npz
[INFO] Dataset ----- No separate flow features (方案A or 方案B)
[INFO] Dataset ----- Loaded 5000 flows
[INFO] X shape: (5000, 128, 35)
[INFO] Time shape: (5000, 128)
[INFO] Mask shape: (5000, 128)
[INFO] sampler mode: inverse-frequency balanced
[INFO] pipeline.py ----- build_dataloaders ----- end

[DEBUG] --TRAIN--- Checking train_loader consistency...
[DEBUG] --TRAIN--- batch 0 flow_ids]: tensor([ 963733993164193, 697556746513573, 1477938102072232,
1168295416756083,

```

```
956219961700596))
[DEBUG --TRAIN--- batch 0 flow_ids]: tensor([2186063401333306, 1648772849361648, 436857490879894,
591378168054443,
3211884173111])

[DEBUG] --TRAIN-trainNoSampler-- Checking train_loader consistency...
[DEBUG --TRAIN-trainNoSampler-- batch 0 flow_ids]: tensor([266605417401827, 266698416238684,
266751598479370, 266753354326044,
266865654217348])
[DEBUG --TRAIN-trainNoSampler-- batch 0 flow_ids]: tensor([273514782515159, 273582103515337,
273596909466898, 273600889479266,
273689329211684])

[DEBUG] --VAL--- Checking VAL_loader consistency...
[DEBUG --VAL--- batch 0 flow_ids]: tensor([966944622115060, 967208114705350, 968866866126547,
970184128707953,
970660937359705])
[DEBUG --VAL--- batch 0 flow_ids]: tensor([1083678247607952, 1083845563393665, 1084166245208978,
1084226195033697,
1084301596426607])

[DEBUG] --TEST--- Checking TEST_loader consistency...
[DEBUG --TEST--- batch 0 flow_ids]: tensor([760085402692707, 760971750835431, 588151797733713,
673474450812107,
677247809890333])
[DEBUG --TEST--- batch 0 flow_ids]: tensor([1720664612595132, 1722692252625006, 1711091367592953,
46544016657700,
50127068521653])

[INFO] Preprocessor Summary:
external_test True
num_train_flows 369161
num_val_flows 52737
num_test_flows 5000
真实训练集类别分布 label_counts_train {'0': 354538, '1': 14623}
label_counts_val {'0': 50716, '1': 2021}
label_counts_test {'0': 2793, '1': 2207}
preprocessor {'packet_numerical_used': ['pkt_len', 'ip_len', 'payload_len', 'ttl_or_hop_limit', 'tcp_window',
'tcp_header_len', 'udp_len'], 'packet_categorical_used': ['ip_version', 'protocol', 'icmp_type', 'icmp_code'],
'packet_binary_used': ['direction', 'has_tcp', 'has_udp', 'has_icmp', 'tcp_fin', 'tcp_syn', 'tcp_rst', 'tcp_psh',
'tcp_ack_flag', 'tcp_urg', 'tcp_ece', 'tcp_cwr'], 'flow_numerical_used': ['flow_duration', 'total_fwd_packets',
'total_backward_packets', 'total_length_of_fwd_packets', 'total_length_of_bwd_packets',
'fwd_packet_length_max', 'fwd_packet_length_min', 'fwd_packet_length_mean', 'fwd_packet_length_std',
'bwd_packet_length_max', 'bwd_packet_length_min', 'bwd_packet_length_mean', 'bwd_packet_length_std',
'flow_bytes_per_s', 'flow_packets_per_s', 'flow_iat_mean', 'flow_iat_std', 'flow_iat_max', 'flow_iat_min',
'fwd_iat_total', 'fwd_iat_mean', 'fwd_iat_std', 'fwd_iat_max', 'fwd_iat_min', 'bwd_iat_total', 'bwd_iat_mean',
'bwd_iat_std', 'bwd_iat_max', 'bwd_iat_min', 'fwd_psh_flags', 'bwd_psh_flags', 'fwd_urg_flags',
'bwd_urg_flags', 'fwd_header_length', 'bwd_header_length', 'fwd_packets_per_s', 'bwd_packets_per_s',
'min_packet_length', 'max_packet_length', 'packet_length_mean', 'packet_length_std',
```



```
'packet_length_variance', 'fin_flag_count', 'syn_flag_count', 'rst_flag_count', 'psh_flag_count',
'ack_flag_count', 'urg_flag_count', 'cwe_flag_count', 'ece_flag_count', 'down_up_ratio',
'average_packet_size', 'avg_fwd_segment_size', 'avg_bwd_segment_size', 'init_win_bytes_forward',
'init_win_bytes_backward', 'act_data_pkt_fwd', 'min_seg_size_forward', 'active_mean', 'active_std',
'active_max', 'active_min', 'idle_mean', 'idle_std', 'idle_max', 'idle_min'], 'flow_categorical_used': [],
'flow_binary_used': ['has_init_win_bytes_forward', 'has_init_win_bytes_backward'], 'packet_log1p_used':
['pkt_len', 'ip_len', 'payload_len', 'tcp_window', 'tcp_header_len', 'udp_len'], 'flow_log1p_used':
['flow_duration', 'total_fwd_packets', 'total_backward_packets', 'total_length_of_fwd_packets',
'total_length_of_bwd_packets', 'fwd_packet_length_max', 'fwd_packet_length_min',
'fwd_packet_length_mean', 'fwd_packet_length_std', 'bwd_packet_length_max', 'bwd_packet_length_min',
'bwd_packet_length_mean', 'bwd_packet_length_std', 'flow_bytes_per_s', 'flow_packets_per_s',
'flow_iat_mean', 'flow_iat_std', 'flow_iat_max', 'flow_iat_min', 'fwd_iat_total', 'fwd_iat_mean', 'fwd_iat_std',
'fwd_iat_max', 'fwd_iat_min', 'bwd_iat_total', 'bwd_iat_mean', 'bwd_iat_std', 'bwd_iat_max', 'bwd_iat_min',
'fwd_header_length', 'bwd_header_length', 'fwd_packets_per_s', 'bwd_packets_per_s', 'min_packet_length',
'max_packet_length', 'packet_length_mean', 'packet_length_std', 'packet_length_variance',
'average_packet_size', 'avg_fwd_segment_size', 'avg_bwd_segment_size', 'init_win_bytes_forward',
'init_win_bytes_backward', 'act_data_pkt_fwd', 'active_mean', 'active_std', 'active_max', 'active_min',
'idle_mean', 'idle_std', 'idle_max', 'idle_min'], 'scaler': 'robust', 'clip_enabled': True, 'clip_lower_quantile':
0.001, 'clip_upper_quantile': 0.999, 'packet_clip_bounds_count': 7, 'flow_clip_bounds_count': 66,
'packet_feature_dim': 35, 'flow_feature_dim': 68, 'input_dim': 35, 'has_flow_features': True}
```

```
=====
[STEP 2] Building model...
```

```
=====
```

[INFO] 方案B - 仅使用Packet特征

[INFO] Input dim: 35

[INFO]-----model --- __init__--fusion_method=None===-d_model=128, nhead=4, num_layers=2,
dim_feedforward=256, dropout=0.2, max_seq_len=128, use_positional_encoding=True,
use_time_encoding=True, use_flow_fusion=False, inject_to_packets=False

[INFO] 方案B - 仅使用Packet特征

[INFO] 输入维度: 35

[INFO] Encoding: Time-Aware (TE(pos+p) from paper)

[INFO] Stage1Transformer.__init__ ***** cls_head_config= 2

```
=====
```

□ Model Statistics:

```
=====
```

Total parameters: 344,963

Trainable parameters: 344,963

Non-trainable params: 0

Parameter breakdown:

```
projection.proj.weight 4,480
projection.proj.bias 128
projection.norm.weight 128
projection.norm.bias 128
encoder.layers.0.self_attn.in_proj_weight 49,152
encoder.layers.0.self_attn.in_proj_bias 384
encoder.layers.0.self_attn.out_proj.weight 16,384
```

encoder.layers.0.self_attn.out_proj.bias 128
encoder.layers.0.linear1.weight 32,768
encoder.layers.0.linear1.bias 256
encoder.layers.0.linear2.weight 32,768
encoder.layers.0.linear2.bias 128
encoder.layers.0.norm1.weight 128
encoder.layers.0.norm1.bias 128
encoder.layers.0.norm2.weight 128
encoder.layers.0.norm2.bias 128
encoder.layers.1.self_attn.in_proj_weight 49,152
encoder.layers.1.self_attn.in_proj_bias 384
encoder.layers.1.self_attn.out_proj.weight 16,384
encoder.layers.1.self_attn.out_proj.bias 128
encoder.layers.1.linear1.weight 32,768
encoder.layers.1.linear1.bias 256
encoder.layers.1.linear2.weight 32,768
encoder.layers.1.linear2.bias 128
encoder.layers.1.norm1.weight 128
encoder.layers.1.norm1.bias 128
encoder.layers.1.norm2.weight 128
encoder.layers.1.norm2.bias 128
encoder.norm.weight 128
encoder.norm.bias 128
attention_pool.attention.0.weight 8,192
attention_pool.attention.0.bias 64
attention_pool.attention.2.weight 64
attention_pool.attention.2.bias 1
attention_pool.fusion.0.weight 49,152
attention_pool.fusion.0.bias 128
attention_pool.fusion.1.weight 128
attention_pool.fusion.1.bias 128
classifier.0.weight 128
classifier.0.bias 128
classifier.2.weight 16,384
classifier.2.bias 128
classifier.5.weight 256
classifier.5.bias 2

Model size: 1.32 MB

=====

=====

[STEP 3] Training model...

=====

[INFO] trainer.py ----- train_model

[train_model----]loss_name = focal

[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):

0.6743782803106809

[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.7472821811603209

[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.7703247961957067

[Epoch 5/500] train_loss=0.037433 val_loss=0.033209 val_accuracy=0.9788 val_f1_label1=0.6917
val_macro_f1=0.8403 val_weighted_f1=0.9776val_auc=0.9808

[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.7957781206083238

[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.8152327276814078

[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.8276470369012766

[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.8357279469151374

[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.8367237762461955

[Epoch 10/500] train_loss=0.032728 val_loss=0.028595 val_accuracy=0.9818 val_f1_label1=0.7416
val_macro_f1=0.8661 val_weighted_f1=0.9810val_auc=0.9850

[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.8409209109525705

[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.8449642911658566

[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.8453729942611392

[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.8491367002415559

[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.8536445602609976

[Epoch 15/500] train_loss=0.031051 val_loss=0.027281 val_accuracy=0.9827 val_f1_label1=0.7624
val_macro_f1=0.8767 val_weighted_f1=0.9822val_auc=0.9873

[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.8557621150059049

[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):

0.8562558247682438
[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.8601244986796285
[Epoch 20/500] train_loss=0.030055 val_loss=0.026268 val_accuracy=0.9829 val_f1_label1=0.7697
val_macro_f1=0.8804 val_weighted_f1=0.9827val_auc=0.9884
[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.8637069948874558
[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.8655498967623446
[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.8673494812218551
[Epoch 25/500] train_loss=0.029158 val_loss=0.025697 val_accuracy=0.9832 val_f1_label1=0.7786
val_macro_f1=0.8849 val_weighted_f1=0.9831val_auc=0.9883
[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.867828099277535
[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.8682565063223938
[Epoch 30/500] train_loss=0.028344 val_loss=0.025477 val_accuracy=0.9831 val_f1_label1=0.7766
val_macro_f1=0.8839 val_weighted_f1=0.9830val_auc=0.9887
[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.8689251259006106
[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.8695191461020033
[Epoch 35/500] train_loss=0.028093 val_loss=0.025458 val_accuracy=0.9834 val_f1_label1=0.7828
val_macro_f1=0.8871 val_weighted_f1=0.9834val_auc=0.9886
[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.870585624826207
[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.8722235179828871
[Epoch 40/500] train_loss=0.027748 val_loss=0.025088 val_accuracy=0.9841 val_f1_label1=0.7895
val_macro_f1=0.8906 val_weighted_f1=0.9840val_auc=0.9889
[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.8722761226123107
[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.8725566883506165
[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/

```

B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.8742974056433168
[Epoch 45/500] train_loss=0.027216 val_loss=0.025110 val_accuracy=0.9841 val_f1_label1=0.7925
val_macro_f1=0.8921 val_weighted_f1=0.9841val_auc=0.9891
[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.874329372181741
[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.8759686726019339
[Epoch 50/500] train_loss=0.026673 val_loss=0.025479 val_accuracy=0.9840 val_f1_label1=0.7909
val_macro_f1=0.8913 val_weighted_f1=0.9840val_auc=0.9890
[Epoch 55/500] train_loss=0.026165 val_loss=0.025267 val_accuracy=0.9844 val_f1_label1=0.7950
val_macro_f1=0.8934 val_weighted_f1=0.9843val_auc=0.9892
[Epoch 60/500] train_loss=0.025797 val_loss=0.025992 val_accuracy=0.9841 val_f1_label1=0.7940
val_macro_f1=0.8929 val_weighted_f1=0.9842val_auc=0.9890
[INFO] early stopping at epoch 60

```

在验证集上寻找最优决策阈值...

验证集上不存在同时满足 $P \geq 0.85$ 、 $R \geq 0.85$ 的阈值

当前最平衡的结果为：

```

{'threshold': np.float64(0.48200000000000003), 'f1': 0.7965346534653466, 'precision_label1':
0.7969291728578505, 'recall_label1': 0.7961405244928254, 'f1_macro': 0.8942154309186464, 'precision':
0.8944029120726809, 'recall': 0.8940281453602229}

```

[THRESHOLD] 最优阈值: 0.48

F1_class1: 0.7965

F1_macro: 0.8942

Precision_macro: 0.8944

Recall_macro: 0.8940

使用阈值 0.482 评估测试集...

[INFO] --- train_model --- 计算F1上限

===== F1 CEILING DIAGNOSIS =====

[VAL ORACLE]

threshold: 0.48263412714004517

precision_label1: 0.7976190476190477

recall_label1: 0.7956457199406235

f1_label1: 0.7966311617537776

tp: 1608

fp: 408

tn: 50308

fn: 413

roc_auc: 0.989023448443914

pr_auc: 0.8759686726019339

num_samples: 52737

num_positive: 2021

num_negative: 50716

[TEST AT VAL THRESHOLD]

threshold: 0.48263412714004517

precision_label1: 0.9555125725338491

recall_label1: 0.22383325781603988

f1_label1: 0.3627019089574156

tp: 494

fp: 23

tn: 2770

fn: 1713

[TEST ORACLE - DIAGNOSTIC ONLY]

threshold: 0.15930141508579254

precision_label1: 0.6672771672771672

recall_label1: 0.9904848210240145

f1_label1: 0.7973737005289075

tp: 2186

fp: 1090

tn: 1703

fn: 21

roc_auc: 0.7707969840453293

pr_auc: 0.7078082564015336

num_samples: 5000

num_positive: 2207

num_negative: 2793

Threshold transfer gap: 0.43467179157149194

=====
Stage1 Transformer - 测试集结果
=====

Loss: 0.6718

F1_label1: 0.3627

Precision_label1: 0.9555

Recall_label1: 0.2238

F1 (Macro): 0.5621

Recall (Macro): 0.6078

Precision (Macro): 0.7867

F1 (Weighted): 0.5854

AUC (OvR Macro): 0.7708

PR_AUC: 0.7078
=====

[RESULT] 测试集 --- 各类别 F1 (降序):

Class_0 : 0.7614

Class_1 : 0.3627

Figure(1400x500)

[INFO] 训练曲线已保存: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/B_external_Wednesday_workingHour/seqLen128headstage1_training_curves.png

Figure(1000x800)

[INFO] 混淆矩阵已保存: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/B_

external_Wednesday_workingHour/seqLen128headstage1_confusion_matrix.png

Figure(1000x600)

[INFO] 各类别 F1 图已保存: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/B_external_Wednesday_workingHour/seqLen128headstage1_per_class_f1.png

[TEST] Full metrics: {'accuracy': 0.6528, 'macro_precision': 0.7867011892336879, 'macro_recall': 0.6077991924597564, 'macro_f1': 0.5620546378212037, 'weighted_precision': 0.7669164951109089, 'weighted_recall': 0.6528, 'weighted_f1': 0.5854187776440396, 'precision_label1': 0.9555125725338491, 'recall_label1': 0.22383325781603988, 'f1_label1': 0.36270190895741555, 'auc': 0.7707969840453293, 'pr_auc': 0.7078082564015336, 'per_class_f1': {'0': 0.7614073666849918, '1': 0.36270190895741555}, 'confusion_matrix': [[2770, 23], [1713, 494]], 'num_classes': 2, 'num_samples': 5000, 'loss': 0.6717914450913668, 'threshold': 0.4820000000000003}

=====
[STEP 4] Exporting embeddings...args

Namespace(packet_csv='/content/drive/MyDrive/s1/data/ar002_et12_20260511_001-stage1_packets.csv', flow_csv='/content/drive/MyDrive/s1/data/ar002_et12_20260511_001-stage1_flows.csv', config='/content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/stage1_config_B_Both.yaml', out_dir='/content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/B_external_Wednesday_workingHour', external_packet_csv='/content/drive/MyDrive/s1/data/Wednesday-workingHours-external5000-stage1_packets.csv', external_flow_csv='/content/drive/MyDrive/s1/data/Wednesday-workingHours-external5000-stage1_flows.csv', export_embeddings=True, use_supcon=False, supcon_weight=None, supcon_temperature=None, supcon_warmup_epochs=None)

=====
[INFO] Exported Stage1 embeddings: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/B_external_Wednesday_workingHour/stage1_train_embeddings.npz

[INFO] Keys: ['flow_id', 'label', 'z', 'split', 'flow_start_timestamp_us', 'source_id', 'destination_id']

[INFO] z shape: (369161, 128)

[INFO] saved embeddings: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/B_external_Wednesday_workingHour/stage1_train_embeddings.npz

[INFO] Stage1 build_stage2_metadata_from_npz npz_path: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/B_external_Wednesday_workingHour/stage1_train_embeddings.npz

[INFO] Exported Stage1 embeddings: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/B_external_Wednesday_workingHour/stage1_val_embeddings.npz

[INFO] Keys: ['flow_id', 'label', 'z', 'split', 'flow_start_timestamp_us', 'source_id', 'destination_id']

[INFO] z shape: (52737, 128)

[INFO] saved embeddings: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/B_external_Wednesday_workingHour/stage1_val_embeddings.npz

[INFO] Stage1 build_stage2_metadata_from_npz npz_path: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/B_external_Wednesday_workingHour/stage1_val_embeddings.npz

[INFO] Exported Stage1 embeddings: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/B_external_Wednesday_workingHour/stage1_test_embeddings.npz

[INFO] Keys: ['flow_id', 'label', 'z', 'split', 'flow_start_timestamp_us', 'source_id', 'destination_id']

[INFO] z shape: (5000, 128)

[INFO] saved embeddings: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/B_external_Wednesday_workingHour/stage1_test_embeddings.npz

[INFO] Stage1 build_stage2_metadata_from_npz npz_path: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/B_external_Wednesday_workingHour/stage1_test_embeddings.npz

[INFO] saved Stage2 flow metadata: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_ex

```

ternal_csv/B_external_Wednesday_workingHour/stage1_flow_metadata.csv
/content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/stage1_config_B_Both.yaml \
--out_dir /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/B_external_Wednesday_workingHour \
--export_embeddings
运行结果: [INFO] 随机种子已设置: 130
[INFO] device=cuda

=====
[STEP 1] Building dataloaders...
=====
[INFO] pipeline.py ----- build_dataloaders --- start
[INFO] pipeline.py ----- build_dataloaders --- split_method chronological
[INFO] data_io.py ----- read_stage1_csvs --- start
[data_io.py] --- read_stage1_csvs--- seed = 130, strategy = head, max_seq_len = 128
[INFO] data_io.py ----- read_stage1_csvs --- end
[INFO] data_io.py ----- read_stage1_csvs --- start
[data_io.py] --- read_stage1_csvs--- seed = 130, strategy = head, max_seq_len = 128
[INFO] data_io.py ----- read_stage1_csvs --- end

=====
=====
[INFO] Split report: chronological_train_val_split_for_external_test
=====
=====
[INFO] train:
num_flows = 369161
label_counts = {0: 354538, 1: 14623}
label1_ratio = 0.039611
time_range = [1778495176848505, 1778495251159359]
[INFO] val:
num_flows = 52737
label_counts = {0: 50716, 1: 2021}
label1_ratio = 0.038322
time_range = [1778495251159598, 1778495261763096]

=====
=====

[INFO] splits.py ----- chronological_train_val_split_for_external_test
每个packet都包含flow信息的方案是 = B
[INFO] data_io.py ----- generate_and_save_stage1_tensors --- start
[data_io.py] packet_x_all shape: (2939475, 35)
[data_io.py] packet_x_all sample:
[[-0.02950557 -0.03237712 -0.03650611 0.5882353 -0.00169567 0.
 0. 1. 0. 0. 0. 0.
 0. 0. 1. 1. 0. 0.
 1. 0. 0. 0. 0. 0.
 1. 0. 0. 0. 0. 0.
 1. 1. 0. 0. 0. ]]

```



```

[data_io.py] 模式: 方案B - 仅Packet特征
[INFO] saved precomputed stage1 tensors: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingH
ours_external_csv/B_external_Wednesday_workingHour/precomputed/seqLen128headtrainB.npz
[INFO] Keys in saved file: ['x', 'time', 'mask', 'labels', 'flow_ids', 'flow_start_timestamp_us', 'source_id',
'destination_id']
[INFO] data_io.py ----- generate_and_save_stage1_tensors_fast --- end
每个packet都包含flow信息的方案是 = B
[INFO] data_io.py ----- generate_and_save_stage1_tensors --- start
[data_io.py] packet_x_all shape: (292982, 35)
[data_io.py] packet_x_all sample:
[[-0.11866869 -0.13415103 -0.5701738 -0.3529412 0.79766387 0.19134793
 0. 1. 0. 0. 0. 0.
 0. 0. 1. 1. 0. 0.
 1. 0. 0. 0. 0. 0.
 1. 0. 0. 0. 1. 0.
 0. 0. 0. 0. 0. ]]
[data_io.py] 模式: 方案B - 仅Packet特征
[INFO] saved precomputed stage1 tensors: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingH
ours_external_csv/B_external_Wednesday_workingHour/precomputed/seqLen128headvalB.npz
[INFO] Keys in saved file: ['x', 'time', 'mask', 'labels', 'flow_ids', 'flow_start_timestamp_us', 'source_id',
'destination_id']
[INFO] data_io.py ----- generate_and_save_stage1_tensors_fast --- end
每个packet都包含flow信息的方案是 = B
[INFO] data_io.py ----- generate_and_save_stage1_tensors --- start
[data_io.py] packet_x_all shape: (66618, 35)
[data_io.py] packet_x_all sample:
[[-0.13217773 -0.12897852 -0.07286663 0.60294116 -0.79766387 -0.8707324
 1.0430166 1. 0. 0. 1. 0.
 0. 0. 0. 1. 0. 0.
 1. 0. 0. 0. 0. 0.
 0. 1. 0. 0. 0. 0.
 0. 0. 0. 0. 0. ]]
[data_io.py] 模式: 方案B - 仅Packet特征
[INFO] saved precomputed stage1 tensors: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingH
ours_external_csv/B_external_Wednesday_workingHour/precomputed/seqLen128headtestB.npz
[INFO] Keys in saved file: ['x', 'time', 'mask', 'labels', 'flow_ids', 'flow_start_timestamp_us', 'source_id',
'destination_id']
[INFO] data_io.py ----- generate_and_save_stage1_tensors_fast --- end
[INFO] Dataset ----- Loading /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external
_csv/B_external_Wednesday_workingHour/precomputed/seqLen128headtrainB.npz
[INFO] Dataset ----- No separate flow features (方案A or 方案B)
[INFO] Dataset ----- Loaded 369161 flows
[INFO] X shape: (369161, 128, 35)
[INFO] Time shape: (369161, 128)
[INFO] Mask shape: (369161, 128)
[INFO] Dataset ----- Loading /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external
_csv/B_external_Wednesday_workingHour/precomputed/seqLen128headvalB.npz

```

```
[INFO] Dataset ----- No separate flow features (方案A or 方案B)
[INFO] Dataset ----- Loaded 52737 flows
[INFO] X shape: (52737, 128, 35)
[INFO] Time shape: (52737, 128)
[INFO] Mask shape: (52737, 128)
[INFO] Dataset ----- Loading /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external
_csv/B_external_Wednesday_workingHour/precomputed/seqLen128headtestB.npz
[INFO] Dataset ----- No separate flow features (方案A or 方案B)
[INFO] Dataset ----- Loaded 5000 flows
[INFO] X shape: (5000, 128, 35)
[INFO] Time shape: (5000, 128)
[INFO] Mask shape: (5000, 128)
[INFO] sampler mode: inverse-frequency balanced
[INFO] pipeline.py ----- build_dataloaders ----- end

[DEBUG] --TRAIN--- Checking train_loader consistency...
[DEBUG --TRAIN--- batch 0 flow_ids]: tensor([ 963733993164193, 697556746513573, 1477938102072232,
1168295416756083,
956219961700596])
[DEBUG --TRAIN--- batch 0 flow_ids]: tensor([2186063401333306, 1648772849361648, 436857490879894,
591378168054443,
3211884173111])

[DEBUG] --TRAIN-trainNoSampler-- Checking train_loader consistency...
[DEBUG --TRAIN-trainNoSampler-- batch 0 flow_ids]: tensor([266605417401827, 266698416238684,
266751598479370, 266753354326044,
266865654217348])
[DEBUG --TRAIN-trainNoSampler-- batch 0 flow_ids]: tensor([273514782515159, 273582103515337,
273596909466898, 273600889479266,
273689329211684])

[DEBUG] --VAL--- Checking VAL_loader consistency...
[DEBUG --VAL--- batch 0 flow_ids]: tensor([966944622115060, 967208114705350, 968866866126547,
970184128707953,
970660937359705])
[DEBUG --VAL--- batch 0 flow_ids]: tensor([1083678247607952, 1083845563393665, 1084166245208978,
1084226195033697,
1084301596426607])

[DEBUG] --TEST--- Checking TEST_loader consistency...
[DEBUG --TEST--- batch 0 flow_ids]: tensor([760085402692707, 760971750835431, 588151797733713,
673474450812107,
677247809890333])
[DEBUG --TEST--- batch 0 flow_ids]: tensor([1720664612595132, 1722692252625006, 1711091367592953,
46544016657700,
50127068521653])

[INFO] Preprocessor Summary:
external_test True
num_train_flows 369161
```

```

num_val_flows 52737
num_test_flows 5000
真实训练集类别分布 label_counts_train {'0': 354538, '1': 14623}
label_counts_val {'0': 50716, '1': 2021}
label_counts_test {'0': 2793, '1': 2207}
preprocessor {'packet_numerical_used': ['pkt_len', 'ip_len', 'payload_len', 'ttl_or_hop_limit', 'tcp_window',
'tcp_header_len', 'udp_len'], 'packet_categorical_used': ['ip_version', 'protocol', 'icmp_type', 'icmp_code'],
'packet_binary_used': ['direction', 'has_tcp', 'has_udp', 'has_icmp', 'tcp_fin', 'tcp_syn', 'tcp_rst', 'tcp_psh',
'tcp_ack_flag', 'tcp_urg', 'tcp_ece', 'tcp_cwr'], 'flow_numerical_used': ['flow_duration', 'total_fwd_packets',
'total_backward_packets', 'total_length_of_fwd_packets', 'total_length_of_bwd_packets',
'fwd_packet_length_max', 'fwd_packet_length_min', 'fwd_packet_length_mean', 'fwd_packet_length_std',
'bwd_packet_length_max', 'bwd_packet_length_min', 'bwd_packet_length_mean', 'bwd_packet_length_std',
'flow_bytes_per_s', 'flow_packets_per_s', 'flow_iat_mean', 'flow_iat_std', 'flow_iat_max', 'flow_iat_min',
'fwd_iat_total', 'fwd_iat_mean', 'fwd_iat_std', 'fwd_iat_max', 'fwd_iat_min', 'bwd_iat_total', 'bwd_iat_mean',
'bwd_iat_std', 'bwd_iat_max', 'bwd_iat_min', 'fwd_psh_flags', 'bwd_psh_flags', 'fwd_urg_flags',
'bwd_urg_flags', 'fwd_header_length', 'bwd_header_length', 'fwd_packets_per_s', 'bwd_packets_per_s',
'min_packet_length', 'max_packet_length', 'packet_length_mean', 'packet_length_std',
'packet_length_variance', 'fin_flag_count', 'syn_flag_count', 'rst_flag_count', 'psh_flag_count',
'ack_flag_count', 'urg_flag_count', 'cwe_flag_count', 'ece_flag_count', 'down_up_ratio',
'average_packet_size', 'avg_fwd_segment_size', 'avg_bwd_segment_size', 'init_win_bytes_forward',
'init_win_bytes_backward', 'act_data_pkt_fwd', 'min_seg_size_forward', 'active_mean', 'active_std',
'active_max', 'active_min', 'idle_mean', 'idle_std', 'idle_max', 'idle_min'], 'flow_categorical_used': [],
'flow_binary_used': ['has_init_win_bytes_forward', 'has_init_win_bytes_backward'], 'packet_log1p_used':
['pkt_len', 'ip_len', 'payload_len', 'tcp_window', 'tcp_header_len', 'udp_len'], 'flow_log1p_used':
['flow_duration', 'total_fwd_packets', 'total_backward_packets', 'total_length_of_fwd_packets',
'total_length_of_bwd_packets', 'fwd_packet_length_max', 'fwd_packet_length_min',
'fwd_packet_length_mean', 'fwd_packet_length_std', 'bwd_packet_length_max', 'bwd_packet_length_min',
'bwd_packet_length_mean', 'bwd_packet_length_std', 'flow_bytes_per_s', 'flow_packets_per_s',
'flow_iat_mean', 'flow_iat_std', 'flow_iat_max', 'flow_iat_min', 'fwd_iat_total', 'fwd_iat_mean', 'fwd_iat_std',
'fwd_iat_max', 'fwd_iat_min', 'bwd_iat_total', 'bwd_iat_mean', 'bwd_iat_std', 'bwd_iat_max', 'bwd_iat_min',
'fwd_header_length', 'bwd_header_length', 'fwd_packets_per_s', 'bwd_packets_per_s', 'min_packet_length',
'max_packet_length', 'packet_length_mean', 'packet_length_std', 'packet_length_variance',
'average_packet_size', 'avg_fwd_segment_size', 'avg_bwd_segment_size', 'init_win_bytes_forward',
'init_win_bytes_backward', 'act_data_pkt_fwd', 'active_mean', 'active_std', 'active_max', 'active_min',
'idle_mean', 'idle_std', 'idle_max', 'idle_min'], 'scaler': 'robust', 'clip_enabled': True, 'clip_lower_quantile':
0.001, 'clip_upper_quantile': 0.999, 'packet_clip_bounds_count': 7, 'flow_clip_bounds_count': 66,
'packet_feature_dim': 35, 'flow_feature_dim': 68, 'input_dim': 35, 'has_flow_features': True}

```

```

=====
[STEP 2] Building model...
=====

```

```

[INFO] 方案B - 仅使用Packet特征
[INFO] Input dim: 35
[INFO]-----model --- __init__--fusion_method=none===-d_model=128, nhead=4, num_layers=2,
dim_feedforward=256, dropout=0.2, max_seq_len=128, use_positional_encoding=True,
use_time_encoding=True, use_flow_fusion=False, inject_to_packets=False
[INFO] 方案B - 仅使用Packet特征
[INFO] 输入维度: 35

```

[INFO] Encoding: Time-Aware (TE(pos+p) from paper)
[INFO] Stage1Transformer.__init__ ***** cls_head_config= 2

=====

□ Model Statistics:

=====

Total parameters: 344,963
Trainable parameters: 344,963
Non-trainable params: 0

Parameter breakdown:

projection.proj.weight 4,480
projection.proj.bias 128
projection.norm.weight 128
projection.norm.bias 128
encoder.layers.0.self_attn.in_proj_weight 49,152
encoder.layers.0.self_attn.in_proj_bias 384
encoder.layers.0.self_attn.out_proj.weight 16,384
encoder.layers.0.self_attn.out_proj.bias 128
encoder.layers.0.linear1.weight 32,768
encoder.layers.0.linear1.bias 256
encoder.layers.0.linear2.weight 32,768
encoder.layers.0.linear2.bias 128
encoder.layers.0.norm1.weight 128
encoder.layers.0.norm1.bias 128
encoder.layers.0.norm2.weight 128
encoder.layers.0.norm2.bias 128
encoder.layers.1.self_attn.in_proj_weight 49,152
encoder.layers.1.self_attn.in_proj_bias 384
encoder.layers.1.self_attn.out_proj.weight 16,384
encoder.layers.1.self_attn.out_proj.bias 128
encoder.layers.1.linear1.weight 32,768
encoder.layers.1.linear1.bias 256
encoder.layers.1.linear2.weight 32,768
encoder.layers.1.linear2.bias 128
encoder.layers.1.norm1.weight 128
encoder.layers.1.norm1.bias 128
encoder.layers.1.norm2.weight 128
encoder.layers.1.norm2.bias 128
encoder.norm.weight 128
encoder.norm.bias 128
attention_pool.attention.0.weight 8,192
attention_pool.attention.0.bias 64
attention_pool.attention.2.weight 64
attention_pool.attention.2.bias 1
attention_pool.fusion.0.weight 49,152
attention_pool.fusion.0.bias 128

attention_pool.fusion.1.weight 128
attention_pool.fusion.1.bias 128
classifier.0.weight 128
classifier.0.bias 128
classifier.2.weight 16,384
classifier.2.bias 128
classifier.5.weight 256
classifier.5.bias 2

Model size: 1.32 MB

[STEP 3] Training model...

[INFO] trainer.py ----- train_model

[train_model----]loss_name = focal

[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.6743782803106809

[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.7472821811603209

[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.7703247961957067

[Epoch 5/500] train_loss=0.037433 val_loss=0.033209 val_accuracy=0.9788 val_f1_label1=0.6917
val_macro_f1=0.8403 val_weighted_f1=0.9776val_auc=0.9808

[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.7957781206083238

[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.8152327276814078

[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.8276470369012766

[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.8357279469151374

[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.8367237762461955

[Epoch 10/500] train_loss=0.032728 val_loss=0.028595 val_accuracy=0.9818 val_f1_label1=0.7416
val_macro_f1=0.8661 val_weighted_f1=0.9810val_auc=0.9850

[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.8409209109525705

[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.8449642911658566

[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.8453729942611392

[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.8491367002415559

[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.8536445602609976

[Epoch 15/500] train_loss=0.031051 val_loss=0.027281 val_accuracy=0.9827 val_f1_label1=0.7624
val_macro_f1=0.8767 val_weighted_f1=0.9822val_auc=0.9873

[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.8557621150059049

[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.8562558247682438

[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.8601244986796285

[Epoch 20/500] train_loss=0.030055 val_loss=0.026268 val_accuracy=0.9829 val_f1_label1=0.7697
val_macro_f1=0.8804 val_weighted_f1=0.9827val_auc=0.9884

[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.8637069948874558

[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.8655498967623446

[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.8673494812218551

[Epoch 25/500] train_loss=0.029158 val_loss=0.025697 val_accuracy=0.9832 val_f1_label1=0.7786
val_macro_f1=0.8849 val_weighted_f1=0.9831val_auc=0.9883

[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.867828099277535

[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.8682565063223938

[Epoch 30/500] train_loss=0.028344 val_loss=0.025477 val_accuracy=0.9831 val_f1_label1=0.7766
val_macro_f1=0.8839 val_weighted_f1=0.9830val_auc=0.9887

[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.8689251259006106


```

[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.8695191461020033
[Epoch 35/500] train_loss=0.028093 val_loss=0.025458 val_accuracy=0.9834 val_f1_label1=0.7828
val_macro_f1=0.8871 val_weighted_f1=0.9834val_auc=0.9886
[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.870585624826207
[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.8722235179828871
[Epoch 40/500] train_loss=0.027748 val_loss=0.025088 val_accuracy=0.9841 val_f1_label1=0.7895
val_macro_f1=0.8906 val_weighted_f1=0.9840val_auc=0.9889
[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.8722761226123107
[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.8725566883506165
[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.8742974056433168
[Epoch 45/500] train_loss=0.027216 val_loss=0.025110 val_accuracy=0.9841 val_f1_label1=0.7925
val_macro_f1=0.8921 val_weighted_f1=0.9841val_auc=0.9891
[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.874329372181741
[INFO] saved best model: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/
B_external_Wednesday_workingHour/seqLen128headstage1_best_model.pt, best score(pr_auc):
0.8759686726019339
[Epoch 50/500] train_loss=0.026673 val_loss=0.025479 val_accuracy=0.9840 val_f1_label1=0.7909
val_macro_f1=0.8913 val_weighted_f1=0.9840val_auc=0.9890
[Epoch 55/500] train_loss=0.026165 val_loss=0.025267 val_accuracy=0.9844 val_f1_label1=0.7950
val_macro_f1=0.8934 val_weighted_f1=0.9843val_auc=0.9892
[Epoch 60/500] train_loss=0.025797 val_loss=0.025992 val_accuracy=0.9841 val_f1_label1=0.7940
val_macro_f1=0.8929 val_weighted_f1=0.9842val_auc=0.9890
[INFO] early stopping at epoch 60

```

在验证集上寻找最优决策阈值...

验证集上不存在同时满足 $P \geq 0.85$ 、 $R \geq 0.85$ 的阈值

当前最平衡的结果为：

```

{'threshold': np.float64(0.48200000000000003), 'f1': 0.7965346534653466, 'precision_label1':
0.7969291728578505, 'recall_label1': 0.7961405244928254, 'f1_macro': 0.8942154309186464, 'precision':
0.8944029120726809, 'recall': 0.8940281453602229}

```

[THRESHOLD] 最优阈值: 0.48

F1_class1: 0.7965

F1_macro: 0.8942

Precision_macro: 0.8944

Recall_macro: 0.8940

使用阈值 0.482 评估测试集...

[INFO] --- train_model --- 计算F1上限

===== F1 CEILING DIAGNOSIS =====

[VAL ORACLE]

threshold: 0.48263412714004517

precision_label1: 0.7976190476190477

recall_label1: 0.7956457199406235

f1_label1: 0.7966311617537776

tp: 1608

fp: 408

tn: 50308

fn: 413

roc_auc: 0.989023448443914

pr_auc: 0.8759686726019339

num_samples: 52737

num_positive: 2021

num_negative: 50716

[TEST AT VAL THRESHOLD]

threshold: 0.48263412714004517

precision_label1: 0.9555125725338491

recall_label1: 0.22383325781603988

f1_label1: 0.3627019089574156

tp: 494

fp: 23

tn: 2770

fn: 1713

[TEST ORACLE - DIAGNOSTIC ONLY]

threshold: 0.15930141508579254

precision_label1: 0.6672771672771672

recall_label1: 0.9904848210240145

f1_label1: 0.7973737005289075

tp: 2186

fp: 1090

tn: 1703

fn: 21

roc_auc: 0.7707969840453293

pr_auc: 0.7078082564015336

num_samples: 5000

num_positive: 2207

num_negative: 2793

Threshold transfer gap: 0.43467179157149194


```

=====
Stage1 Transformer - 测试集结果
=====
Loss: 0.6718
F1_label1: 0.3627
Precision_label1: 0.9555
Recall_label1: 0.2238
F1 (Macro): 0.5621
Recall (Macro): 0.6078
Precision (Macro): 0.7867
F1 (Weighted): 0.5854
AUC (OvR Macro): 0.7708
PR_AUC: 0.7078
=====

[RESULT] 测试集 --- 各类别 F1 (降序):
Class_0 : 0.7614
Class_1 : 0.3627
Figure(1400x500)
[INFO] 训练曲线已保存: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/B_
external_Wednesday_workingHour/seqLen128headstage1_training_curves.png
Figure(1000x800)
[INFO] 混淆矩阵已保存: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/B_
external_Wednesday_workingHour/seqLen128headstage1_confusion_matrix.png
Figure(1000x600)
[INFO] 各类别 F1 图已保存: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv
/B_external_Wednesday_workingHour/seqLen128headstage1_per_class_f1.png

[TEST] Full metrics: {'accuracy': 0.6528, 'macro_precision': 0.7867011892336879, 'macro_recall':
0.6077991924597564, 'macro_f1': 0.5620546378212037, 'weighted_precision': 0.7669164951109089,
'weighted_recall': 0.6528, 'weighted_f1': 0.5854187776440396, 'precision_label1': 0.9555125725338491,
'recall_label1': 0.22383325781603988, 'f1_label1': 0.36270190895741555, 'auc': 0.7707969840453293,
'pr_auc': 0.7078082564015336, 'per_class_f1': {'0': 0.7614073666849918, '1': 0.36270190895741555},
'confusion_matrix': [[2770, 23], [1713, 494]], 'num_classes': 2, 'num_samples': 5000, 'loss':
0.6717914450913668, 'threshold': 0.4820000000000003}

=====
[STEP 4] Exporting embeddings...args
Namespace(packet_csv='/content/drive/MyDrive/s1/data/ar002_et12_20260511_001-stage1_packets.csv',
flow_csv='/content/drive/MyDrive/s1/data/ar002_et12_20260511_001-stage1_flows.csv', config='/content
/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/stage1_config_B_Both.yaml', out_d
ir='/content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/B_external_Wednesda
y_workingHour', external_packet_csv='/content/drive/MyDrive/s1/data/Wednesday-workingHours-extern
al5000-stage1_packets.csv', external_flow_csv='/content/drive/MyDrive/s1/data/Wednesday-workingHou
rs-external5000-stage1_flows.csv', export_embeddings=True, use_supcon=False, supcon_weight=None,
supcon_temperature=None, supcon_warmup_epochs=None)
=====
[INFO] Exported Stage1 embeddings: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_
external_csv/B_external_Wednesday_workingHour/stage1_train_embeddings.npz
[INFO] Keys: ['flow_id', 'label', 'z', 'split', 'flow_start_timestamp_us', 'source_id', 'destination_id']

```

```
[INFO] z shape: (369161, 128)
[INFO] saved embeddings: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/B_external_Wednesday_workingHour/stage1_train_embeddings.npz
[INFO] Stage1 build_stage2_metadata_from_npz npz_path: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/B_external_Wednesday_workingHour/stage1_train_embeddings.npz
[INFO] Exported Stage1 embeddings: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/B_external_Wednesday_workingHour/stage1_val_embeddings.npz
[INFO] Keys: ['flow_id', 'label', 'z', 'split', 'flow_start_timestamp_us', 'source_id', 'destination_id']
[INFO] z shape: (52737, 128)
[INFO] saved embeddings: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/B_external_Wednesday_workingHour/stage1_val_embeddings.npz
[INFO] Stage1 build_stage2_metadata_from_npz npz_path: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/B_external_Wednesday_workingHour/stage1_val_embeddings.npz
[INFO] Exported Stage1 embeddings: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/B_external_Wednesday_workingHour/stage1_test_embeddings.npz
[INFO] Keys: ['flow_id', 'label', 'z', 'split', 'flow_start_timestamp_us', 'source_id', 'destination_id']
[INFO] z shape: (5000, 128)
[INFO] saved embeddings: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/B_external_Wednesday_workingHour/stage1_test_embeddings.npz
[INFO] Stage1 build_stage2_metadata_from_npz npz_path: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/B_external_Wednesday_workingHour/stage1_test_embeddings.npz
[INFO] saved Stage2 flow metadata: /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/B_external_Wednesday_workingHour/stage1_flow_metadata.csv
详细解析这个结果。
```

助手 2026-07-27 10:39:48

总体结论

这次方案 B 在 `Wednesday-workingHours-external5000` 上表现明显好于之前的 `pbx23` 外部集：

- ROC-AUC: ****0.7708****，说明模型具有中等程度的跨域排序能力；
- PR-AUC: ****0.7078****，明显高于该测试集 44.14% 的正样本基线；
- 但是验证集阈值 `0.4826` 迁移到外部集后过于保守：
- Precision: ****95.55%****
- Recall: ****22.38%****
- 漏掉 ****1,713/2,207**** 个异常。

因此，这次不是“模型完全不会识别外部异常”，而是：

> 模型能够在 Wednesday 数据中区分异常与正常，但输出分数整体发生了明显偏移，导致从 ar002 验证集得到的阈值不适用于外部域。

这是“有跨域判别能力，但阈值/概率校准失败”的典型结果。

1. 数据规模与类别分布

数据集	流数	正常	异常	异常比例
Train	369,161	354,538	14,623	3.96%

```
| Validation | 52,737 | 50,716 | 2,021 | 3.83% |  
| Wednesday test | 5,000 | 2,793 | 2,207 | 44.14% |
```

外部测试集与训练集类别先验差异非常大：

```
3.96%  
44.14%
```

这会显著影响：

- Precision;
- PR-AUC;
- 最优分类阈值;
- 输出概率校准;
- accuracy 和 weighted-F1。

因此，本次外部 PR-AUC 不能直接和之前异常率仅 5.62% 的 `pbx23` PR-AUC 进行绝对比较；PR-AUC 本身会随正样本比例变化。ROC-AUC 受类别比例影响较小，更适合跨外部集比较。

外部数据共有 66,618 个 packet、5,000 个 flow，平均：

```
66,618 / 5,000 ≈ 13.32 packets/flow
```

训练集平均约 7.96 packets/flow，说明外部流的序列长度结构仍存在明显差异。

2. 模型设置

本次实际运行的是方案 B：

```
packet  
packet dimension = 35  
max_seq_len = 128  
fusion_method = none  
use_flow_fusion = False
```

模型配置：

```
| 参数 | 数值 |  
|---|---|  
| d_model | 128 |  
| Transformer 层数 | 2 |  
| Attention heads | 4 |  
| Feed-forward dimension | 256 |  
| Dropout | 0.2 |  
| 最大序列长度 | 128 |  
| 参数量 | 344,963 |  
| 模型大小 | 1.32 MB |  
| Loss | Focal loss |  
| Sampler | inverse-frequency balanced |
```

虽然 Preprocessor Summary 中存在：

```
flow_feature_dim = 68  
has_flow_features = True
```

但 NPZ 不包含 `flow\_feats`，Dataset 明确显示 `No separate flow features`，模型也设置了 `use\_flow\_fusion=False`。因此这些 flow 特征没有实际进入模型，只是预处理器元数据的日志表达不够准确。

3. 训练过程

训练在 epoch 60 触发 early stopping。

Epoch	Train loss	Val loss	Val class-1 F1	Val AUC
---:	---:	---:	---:	---:
5	0.03743	0.03321	0.6917	0.9808
15	0.03105	0.02728	0.7624	0.9873
25	0.02916	0.02570	0.7786	0.9883
35	0.02809	0.02546	0.7828	0.9886
45	0.02722	0.02511	0.7925	0.9891
55	0.02617	0.02527	0.7950	0.9892
60	0.02580	0.02599	0.7940	0.9890

训练过程正常：

- train loss 平稳下降；
- validation loss 在 epoch 40 后进入平台期；
- validation AUC 稳定在 0.989 左右；
- 没有训练发散；
- 后期只有轻微过拟合；
- 最佳模型根据验证 PR-AUC 保存，而不是直接使用最后一轮。

最佳验证 PR-AUC：

0.8759686726

值得注意的是，这一训练轨迹和上一次方案 B 的结果完全一致。这是合理的，因为：

- 训练 CSV 相同；
- 验证 CSV 相同；
- 配置相同；
- seed 相同；
- 只替换了外部测试集。

这也侧面说明随机种子和训练过程具有良好可复现性。

4. 验证集表现

精确最佳验证阈值：

threshold = 0.482634

验证集混淆矩阵：

50308	408
413	1608

对应：

指标	数值
---	---:
Precision class 1	79.76%
Recall class 1	79.56%

```
| F1 class 1 | 79.66% |
| ROC-AUC | 0.9890 |
| PR-AUC | 0.8760 |
| 正常误报率 | 0.80% |
| 异常漏报率 | 20.44% |
```

验证集上 Precision 和 Recall 很平衡，但不存在两者都达到 85%的阈值。

这说明模型在训练域内部的实际 class-1 F1 上限约为 0.797。

5. 外部集在验证阈值下的表现

使用验证集阈值：

```
threshold = 0.482634
```

外部混淆矩阵：

2770	23
1713	494

异常类指标

```
Precision = 494 / (494 + 23) = 95.55%
Recall    = 494 / (494 + 1713) = 22.38%
F1        = 36.27%
```

模型一共发出 517 个异常告警，其中：

- 494 个是真异常；
- 23 个是误报。

这说明告警非常“纯”，但模型只报告最确定的一小部分异常。

正常类表现

正常流误报率：

```
23 / 2793 = 0.824%
```

Specificity:

```
2770 / 2793 = 99.18%
```

因此，在当前阈值下，模型工作在：

> 极低误报、极高精确率、极低异常召回率
的保守运行点。

6. Accuracy 与整体指标

测试结果：

```
| 指标 | 数值 |
|---|---:|
| Accuracy | 65.28% |
| Macro precision | 78.67% |
```

```
| Macro recall | 60.78% |
| Macro F1 | 56.21% |
| Weighted F1 | 58.54% |
| Class-0 F1 | 76.14% |
| Class-1 F1 | 36.27% |
```

这次 accuracy 具有一定参考价值，因为测试集不像之前那样极度不平衡。

如果全部预测为正常，accuracy 为：

```
2793 / 5000 = 55.86%
```

当前模型：

```
65.28%
```

高于多数类基线约 9.42 个百分点。

但 accuracy 仍没有充分体现模型的排序能力，因为当前主要问题是阈值过高，而不是表示空间完全失效。

Macro recall 为：

```
( 99.18% + 22.38% ) / 2
= 60.78%
```

它实际上就是二分类中的 balanced accuracy，说明当前固定阈值下整体性能中等，但严重偏向正常类。

7. ROC-AUC 与 PR-AUC是本次最关键的结果

```
## ROC-AUC = 0.7708
```

含义是：

> 随机抽取一个外部异常流和一个外部正常流，模型约有 77.08%的概率给异常流更高的异常分数。

这已经显著高于随机水平 0.5，说明 packet embedding 和分类器确实学到了一部分能迁移到 Wednesday 数据的攻击特征。

它与此前 `pbx23` 的结果形成鲜明对比：

```
| 外部数据 | ROC-AUC |
|---|---|
| pbx23 | 0.4469 |
| Wednesday external5000 | 0.7708 |
```

因此不能笼统地说“方案 B 没有跨域能力”。更准确的结论是：

> 跨域效果高度依赖目标数据集；模型能迁移到 Wednesday 的部分攻击模式，但不能迁移到 pbx23 的异常模式。

```
## PR-AUC = 0.7078
```

外部集异常比例为 44.14%，随机排序的 PR-AUC 基线约为：

```
0.4414
```

实际 PR-AUC：

```
0.7078
```

明显高于随机基线，证明异常排序是有效的。

但要注意：由于本次正样本率高达 44.14%，不能把 0.7078 直接与 pbx23 的 0.0474 做倍数比较。

8. 为什么外部 F1 低，但 AUC 很好？

这两个结果并不矛盾：

```
ROC-AUC = 0.7708
PR-AUC  = 0.7078
F1       = 0.3627
```

原因是：

- AUC 衡量所有可能阈值下的排序能力；
- F1 只衡量当前阈值 `0.4826`；
- 模型排序尚可；
- 但外部样本的分数整体比验证集低；
- 所以大量真实异常没有跨过 0.4826。

可以概括为：

/

这和上一次 pbx23 不同。pbx23 是 AUC 本身都低于 0.5，说明排序失败；Wednesday 是排序有效但阈值失配。

9. 外部 Oracle 阈值分析

测试集标签参与搜索后，诊断性最佳阈值为：

```
threshold = 0.159301
```

结果：

```
TP = 2186
FP = 1090
TN = 1703
FN = 21
```

对应：

```
| 指标 | 数值 |
|---|---|
| Precision class 1 | 66.73% |
| Recall class 1 | 99.05% |
| F1 class 1 | 79.74% |
| 正常误报率 | 39.03% |
| 异常漏报率 | 0.95% |
```

这证明了一个重要事实：

> 绝大多数外部异常的分数仍然高于某个合理水平，只是整体分数没有达到训练域确定的 0.4826。

但是这个 oracle 阈值不能作为正式测试结果，因为它使用了测试集真实标签来选择阈值，存在测试信息泄漏。它只能用于诊断模型潜力。

而且这个阈值虽然异常召回率极高，但正常误报率高达：

```
1090 / 2793 = 39.03%
```

对于现实 NIDS 通常过高。

因此 `0.1593` 是 “最大化异常类 F1” 的数学最优点，不一定是部署最优点。

10. Threshold transfer gap 的含义

日志给出：

```
Threshold transfer gap = 0.4346718
```

这个数值实际上是：

```
oracle F1 -          F1
= 0.79737 - 0.36270
= 0.43467
```

它不是两个阈值相减：

```
0.4826 - 0.1593 ≠ 0.4347
```

因此这个字段更准确的名字应当是：

```
test_f1_recoverable_gap
```

或者：

```
threshold_transfer_f1_gap
```

该差距很大，说明通过目标域校准，理论上存在明显的性能恢复空间。

11. 与 pbx23 外部结果比较

两次采用的是相同方案 B、相同训练集、相同模型 checkpoint，区别只是外部测试域。

指标 pbx23 Wednesday
--- --- ---
样本数 1,263 5,000
正样本率 5.62% 44.14%
ROC-AUC 0.4469 0.7708
PR-AUC 0.0474 0.7078
验证阈值 Precision 2.86% 95.55%
验证阈值 Recall 2.82% 22.38%
验证阈值 F1 2.84% 36.27%
Oracle F1 12.06% 79.74%

这说明：

1. Wednesday 中存在与训练域相似的 packet 行为模式；
2. pbx23 的异常模式与训练域差异更大；
3. 模型的跨域泛化不是统一的，而是依赖目标攻击类型和网络环境；
4. Wednesday 的主要问题是 calibration shift；
5. pbx23 的主要问题是 representation/ranking shift。

12. 外部5000数据的抽样方式需要核查

文件名为：

而类别分布接近人为平衡：

```
2793
2207
```

需要确认这 5,000 条是怎样选出来的：

- 随机抽样；
- 分层抽样；
- 按攻击类别均衡抽样；
- 连续时间窗口；
- 正常/攻击分别抽取后拼接。

如果它是人为分层构建的评估集，那么：

- ROC-AUC仍然有意义；
- Recall仍然有意义；
- PR-AUC和Precision会受到人工正样本率影响；
- 65.28%的accuracy不能代表真实网络流量下的部署准确率；
- 95.55%的Precision也不能直接视为生产环境Precision。

真实环境如果异常率远低于44%，即使TPR/FPR相同，Precision也会明显下降。

例如当前运行点：

```
TPR = 22.38%
FPR = 0.824%
```

假设真实异常率只有1%，估算Precision约为：

```
(0.2238 × 0.01) /
[(0.2238 × 0.01) + (0.00824 × 0.99)]
= 21.5%
```

所以测试集里的95.55% Precision主要受高异常比例影响，不能直接作为线上告警纯度。

13. 如何选择更实用的阈值

不要直接采用测试 oracle 的 0.1593。建议从 Wednesday 数据中单独划出一个小型 calibration 集，用它确定阈值，再在剩余 Wednesday 数据上做最终测试。

阈值应根据业务目标选择，例如：

低误报模式

要求：

```
FPR ≤ 1%
```

当前验证阈值已经接近这个目标：

```
FPR = 0.824%
Recall = 22.38%
```

适合高置信度报警，但漏报严重。

平衡模式

选择最大化：

```
macro-F1
```

```
Youden J = TPR - FPR
```

通常会比0.4826更低，但不会像0.1593一样把召回率推到99%。

高召回模式

设定：

```
Recall ≥ 90%
```

然后在满足条件的阈值中选择最高Precision。不过需要接受显著增加的误报。

对于 NIDS，建议报告完整的运行点表：

```
| 目标 | Threshold | Recall | Precision | FPR |
|---|---|---|---|---|
| FPR ≤ 0.1% | 待计算 | 待计算 | 待计算 | ≤0.1% |
| FPR ≤ 1% | 待计算 | 待计算 | 待计算 | ≤1% |
| FPR ≤ 5% | 待计算 | 待计算 | 待计算 | ≤5% |
| Recall ≥ 80% | 待计算 | ≥80% | 待计算 | 待计算 |
| Max macro-F1 | 待计算 | 待计算 | 待计算 | 待计算 |
```

这比只报告单一最大F1阈值更符合入侵检测论文和部署需求。

14. Loss较高的原因

外部测试 loss：

```
0.6718
```

显著高于验证 loss：

```
0.025
```

但 AUC 仍有0.7708。

这是因为 loss 不仅评价排序，还强烈惩罚概率置信度。外部大量异常被模型赋予低于0.4826的分数，造成：

- 排序大体正确；
- 绝对概率位置错误；
- loss 很高。

因此高 test loss 与中等 AUC 可以同时出现，再次说明概率校准/域偏移明显。

15. Embedding 导出

三份 embedding 均导出成功：

```
| 文件 | Shape |
|---|---|
| train embeddings | `(369161, 128)` |
| validation embeddings | `(52737, 128)` |
| Wednesday test embeddings | `(5000, 128)` |
```

包含：

```
flow_id
label
z
```

```
split
flow_start_timestamp_us
source_id
destination_id
```

同时生成了 Stage 2 metadata。

这些 embedding 可以进入 Stage 2，但建议先检查：

- train、validation、Wednesday 是否按域明显分群；
- Wednesday 正负样本是否在 embedding 空间可分；
- Stage 2 是否能恢复低分但排序正确的异常；
- Stage 2 的阈值是否也存在相同域迁移问题。

最终评价

本次结果比之前的外部测试有实质性改善：

> 方案 B 对 Wednesday 外部数据学到了可迁移的异常排序信息，ROC-AUC 0.7708、PR-AUC 0.7078，证明模型不是随机分类器。

但当前验证阈值不适用于外部域：

> 它以仅0.824%的正常误报率换取了95.55%的告警Precision，但异常Recall只有22.38%，漏掉了1,713个异常。

所以当前模型适合描述为：

下一步最重要的工作不是重新训练模型，而是先保留一个独立的 Wednesday calibration 子集，绘制 score 分布、ROC/PR曲线和不同FPR约束下的运行点。如果目标域校准后仍能在独立测试部分保持接近0.77的ROC-AUC和较高F1，才能确认方案 B 对该数据域具有稳定的外部泛化能力。

用户 2026-07-27 13:51:10

Files mentioned by the user:

```
## =====
=====...:
```

C:\Users\XiaoyanXiong\codex/attachments/ee468c9e-89e6-4ad3-8495-b32786603f81/pasted-text.txt

My request for Codex:

```
!python /content/drive/MyDrive/s2/run_stage2.py \
--stage1_dir /content/drive/MyDrive/s1/0727/0727_Wednesday_workingHours_external_csv/B_external_
Wednesday_workingHour \
--config /content/drive/MyDrive/s2/0727/stage2_config_used.yaml \
--out_dir /content/drive/MyDrive/s2/0727/B_external_Wednesday_workingHours_leaky_first_50/ \
--context_method source_host \
--context_policy online \
--window_size 128 \
--seed 130 \
--epochs 500 \
```

--batch_size 128 \
运行结果，详细解析

助手 2026-07-27 13:51:18

我会先读取 Stage 2 的完整日志，核对上下文窗口构造、样本泄漏风险、训练/验证/外部指标及阈值来源，再结合刚才 Stage 1 的 Wednesday 结果判断 Stage 2 到底带来了真实增益还是上下文捷径。

助手 2026-07-27 13:51:26

日志已明确标记这是一个“故意泄漏的负对照实验”：训练目标额外混入了 50% 的验证行和 50% 的测试行。Stage 2 的外部 AUC/PR-AUC 显著上升，但不能作为泛化结果。接下来我会把泄漏方式、上下文构造和相对 Stage 1 的表面增益逐项量化。

助手 2026-07-27 13:53:16

总体结论

这次 Stage 2 的数值非常漂亮：

- 外部 ROC-AUC：从 Stage 1 的 **0.7708** 提升到 **0.9471**
- 外部 PR-AUC：从 **0.7078** 提升到 **0.9115**
- class-1 F1：从 **0.3627** 提升到 **0.6613**
- class-1 Recall：从 **22.38%** 提升到 **51.74%**

但是，这些结果不能作为 Stage 2 的真实外部泛化性能，因为日志明确说明：

```
This run intentionally introduces train/val/test leakage.  
Use it only as a negative-control demonstration.
```

训练数据中主动混入了：

```
50% validation = 26,368  
50% test       = 2,500
```

因此，这是一个成功的“泄漏负对照实验”，不是合法的独立测试实验。

它能证明：

> 当 Stage 2 获得目标域的部分有标签样本后，source-host 上下文模型能够显著适应 Wednesday 数据。

但不能证明：

> Stage 2 能在完全未见过的 Wednesday 数据上达到 AUC 0.947、PR-AUC 0.912。

1. Stage 2 输入数据

Stage 2 加载了 Stage 1 导出的128维 embedding：

Split	样本数	正常	异常	异常率
--- ---:	---	---	---	---
Train	369,161	354,538	14,623	3.96%
Validation	52,737	50,716	2,021	3.83%

| Test | 5,000 | 2,793 | 2,207 | 44.14% |

总 embedding 数:

```
369161 + 52737 + 5000 = 426898
z shape = (426898, 128)
```

这与日志完全一致。

2. 上下文构造方式

本次参数是:

```
context_method = source_host
context_policy = online
window_size = 128
endpoint_mode = same_endpoint
deduplicate = True
include_target = True
```

含义大致是:

- 按源主机组织网络流上下文;
- 对每个目标 flow, 查找同一端点的历史 flow;
- online 策略原则上只使用当前时刻及此前可见信息;
- 最多保留128个位置;
- 重复 flow 被去除;
- 当前目标 flow 本身也进入模型结构。

模型使用目标 embedding 作为 query, 通过多头注意力读取该源主机的上下文 embedding, 随后经过 gated fusion 和分类头预测标签。

从日志看, `clean\_context=True` 表示上下文索引构造本身使用正常的 ContextIndexBuilder, 而不是直接根据标签选择上下文。也就是说:

> 泄漏主要发生在“哪些目标样本参与训练”, 不是 ContextIndexBuilder 直接把标签塞进上下文。

但这并不能消除测试泄漏, 因为2,500条测试目标已经带着标签进入训练。

3. 上下文长度分析

按 split 统计的历史长度:

```
| Split | 平均长度 | 中位数 | P90 | 最大值 |
|---|---:|---:|---:|---:|
| Train | 81.11 | 127 | 127 | 127 |
| Validation | 99.06 | 127 | 127 | 127 |
| Test | 108.63 | 127 | 127 | 127 |
```

中位数、P90、P95 都达到127, 说明大量样本的上下文已经达到窗口上限。

这意味着:

- `window\_size=128` 经常饱和;
- 活跃主机有超过128条历史流;
- 模型看到的通常是最近127条历史加目标流;

- 窗口从128继续增大可能仍带来信息，但显存和计算量会增加；
- 当前结果可能对高活跃 source host 特别有利。

日志中还有两个看似矛盾的值：

```
Context Diagnostics min = 0
context_lengths min = 1
```

合理解释是：

- diagnostics 统计 “历史记录数”，允许为0；
- 模型输入长度包含 target 本身，因此最少为1。

建议以后分别打印：

```
history_length
total_sequence_length_including_target
```

4. 上下文长度本身与标签高度相关

不同标签的平均历史长度：

Split	正常平均长度	异常平均长度
Train	80.22	102.86
Validation	98.31	117.95
Test	101.88	117.16

无历史比例：

Split	正常	异常
Train	5.70%	3.13%
Validation	2.18%	1.63%
Test	0.54%	0.09%

异常流通常拥有更长的同源主机历史。这可能是合理的安全信号，例如攻击主机短时间内持续产生大量连接；但也可能成为数据集捷径：

- > 模型可能部分通过“这个主机最近是否非常活跃”判断异常，而不完全依赖流量内容。

建议后续做以下消融：

- 固定所有样本的上下文长度；
- 显式屏蔽 padding 数量信息；
- 打乱同标签内的上下文；
- 使用随机主机上下文；
- target-only，不使用历史；
- window size分别使用16、32、64、128。

如果随机上下文仍能提升很多，说明模型可能主要利用长度或数据域特征，而非真正的流序列关系。

5. 泄漏具体是怎样发生的

日志明确写道：

```

fraction = 0.500
selection = first
leaked_counts = {
    val: 26368,
    test: 2500
}

```

最终训练目标由以下部分组成：

```

train 369,161
val    26,368
test   2,500
      398,029

```

训练 loader 统计：

```

n0 = 382,018
n1 = 16,011

```

合计也是：

```

382018 + 16011 = 398029

```

因此泄漏确实进入了训练目标，不只是上下文中无标签地看到 embedding。

“first 50%” 的额外问题

`selection=first` 很可能表示选择验证集和测试集按当前顺序排列的前50%。

如果数据已经按时间排序，那么训练模型看到的是：

- Wednesday 前半段测试目标及标签；
- 然后在全体 Wednesday 5,000条上评估。

这相当于一种有监督目标域适配，但评估仍包含训练过的前2,500条，因此不是合法的“前半段适配、后半段测试”。

正确的有监督域适配实验应该是：

```

2500 target calibration/adaptation
2500 final test

```

并且最终指标只能在后2,500条上计算。

当前是在全部5,000条上评估，其中一半已经作为训练目标出现。

6. 泄漏还会通过上下文扩散

即使只把前2,500个 test target 加入训练，后2,500个 test 样本也可能受间接影响：

- online 上下文会使用此前同一 source host 的流；
- 后半段测试样本的历史中可能包含前半段测试流；
- 前半段流的标签已经参与模型训练；
- 模型已经对 Wednesday 域、攻击模式和主机活动模式进行了适配。

使用历史 embedding 本身并不一定违规——真实在线系统也能看到历史流。但模型参数已经通过这些目标域流的标签更新，这属于明确的有监督域适配。

7. Stage 2 模型结构

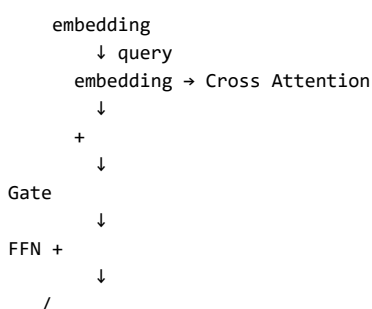
模型为 `Stage2TargetQueryGatedAttention`：

- 输入 embedding: 128维;
- query: 当前目标流;
- history: 同 source host 历史流;
- cross-attention heads: 4;
- FFN维度: 256;
- dropout: 0.25;
- age positional encoding;
- gated context fusion;
- 两分类输出。

模型参数:

283,522
FP32 1.082 MB

结构逻辑是:



这个结构适合捕捉:

- 扫描行为;
- 同一源短时间大量连接;
- 重复失败访问;
- 连续攻击阶段;
- 单流不明显、上下文聚合后明显的行为。

8. 训练配置

主要训练参数:

参数	数值
Learning rate	3e-5
Weight decay	3e-4
Patience	15
最大 epochs	500
实际 epochs	76
最佳 epoch	61
Batch size	128
选择最佳模型指标	PR-AUC
Focal gamma	1.0
Label smoothing	0
Manual alpha	[1.0, 1.01]
目标采样正样本率	5%

训练集真实正样本率:

```
16011 / 398029 ≈ 4.02%
```

sampler将其轻微提高到5%，不是非常激进的重采样。

Focal loss的`gamma=1.0`也比较温和；`alpha=[1.0,1.01]`几乎没有额外类别权重。

这套配置比 Stage 1 的 inverse-frequency balanced sampler保守很多。

9. 训练过程

训练在 epoch 76 early stopping，最佳 checkpoint 是 epoch 61:

```
best_epoch = 61
best_val PR-AUC ≈ 0.8990
```

训练耗时:

```
5862.59 ≈ 97.71
epoch 77.14
```

主要趋势:

Epoch	Train loss	Val loss	Val class-1 F1	Val PR-AUC	Val AUC
1	0.0557	0.0241	0.7922	0.8595	0.9855
10	0.0232	0.0203	0.8012	0.8867	0.9916
20	0.0219	0.0206	0.8084	0.8886	0.9921
40	0.0207	0.0201	0.8147	0.8940	0.9927
53	0.0200	0.0198	0.8155	0.8971	0.9928
61	0.0193	0.0192	0.8161	0.8990	0.9930
76	0.0190	0.0202	0.8199	0.8970	0.9925

训练过程稳定，没有发散。

但验证集本身有50%的样本进入训练，所以:

- validation PR-AUC 被污染;
- early stopping 选择也受到泄漏影响;
- best epoch 61不能视为在独立验证集上选出的结果;
- validation阈值也不是严格独立确定的。

10. 验证结果

自动选择阈值:

```
threshold = 0.470303
```

混淆矩阵:

50276	440
324	1697

指标:

指标	数值
---	---

```
| Accuracy | 98.55% |
| Class-1 Precision | 79.41% |
| Class-1 Recall | 83.97% |
| Class-1 F1 | 81.63% |
| Macro-F1 | 90.44% |
| ROC-AUC | 0.9930 |
| PR-AUC | 0.8990 |
```

相对于 Stage 1 验证结果:

```
| 指标 | Stage 1 | Stage 2 leaky | 变化 |
|---|---|---|---|
| ROC-AUC | 0.9890 | 0.9930 | +0.0040 |
| PR-AUC | 0.8760 | 0.8990 | +0.0231 |
| Class-1 F1 | 0.7966 | 0.8163 | +0.0196 |
| Class-1 Recall | 0.7956 | 0.8397 | +0.0441 |
```

这个提升不大，但不能作为正式增益，因为半个验证集参与了训练。

11. 外部测试结果

使用验证阈值 `0.470303`:

2688	105
1065	1142

指标:

```
| 指标 | 数值 |
|---|---|
| Accuracy | 76.60% |
| Class-1 Precision | 91.58% |
| Class-1 Recall | 51.74% |
| Class-1 F1 | 66.13% |
| Macro-F1 | 74.13% |
| ROC-AUC | 0.9471 |
| PR-AUC | 0.9115 |
| 正常误报率 | 3.76% |
| 异常漏报率 | 48.26% |
```

与 Stage 1 相比:

```
| 指标 | Stage 1 B | Stage 2 leaky | 表面变化 |
|---|---|---|---|
| Accuracy | 65.28% | 76.60% | +11.32 pp |
| ROC-AUC | 0.7708 | 0.9471 | +0.1763 |
| PR-AUC | 0.7078 | 0.9115 | +0.2037 |
| Class-1 Precision | 95.55% | 91.58% | -3.97 pp |
| Class-1 Recall | 22.38% | 51.74% | +29.36 pp |
| Class-1 F1 | 36.27% | 66.13% | +29.86 pp |
| Macro-F1 | 56.21% | 74.13% | +17.92 pp |
```

这些增益非常大，但混合了至少三个来源：

1. Stage 2 source-host 上下文的真实贡献；
2. 使用50% Wednesday测试标签进行域适配的贡献；
3. 在已参与训练的2,500条测试样本上重新评估产生的直接乐观偏差。

所以不能把29.86个百分点的F1提升全部归功于Stage 2上下文。

12. Test Oracle结果

测试标签参与阈值选择后：

```
oracle threshold = 0.034747
```

混淆矩阵：

	2258	535
1	2206	

结果：

```
| 指标 | 数值 |
|---|---|
| Precision class 1 | 80.48% |
| Recall class 1 | 99.95% |
| F1 class 1 | 89.17% |
| Accuracy | 89.28% |
| Macro-F1 | 89.28% |
| ROC-AUC | 0.9471 |
| PR-AUC | 0.9115 |
```

异常几乎全部被检出：

```
2206 / 2207
```

但正常误报率为：

```
535 / 2793 = 19.15%
```

这个 oracle 阈值不能用于正式测试，因为它使用了测试标签。

13. 阈值仍然存在严重跨域迁移问题

验证阈值：

```
0.470303
```

测试 oracle 阈值：

```
0.034747
```

二者相差超过一个数量级。

日志给出的：

```
THRESHOLD TRANSFER GAP = 0.230411
```

是F1差距：

```
0.891673 - 0.661262 = 0.230411
```

而不是阈值数值之差。

这说明即使加入泄漏目标域数据之后，模型输出概率在验证域和Wednesday域之间仍严重不一致：

- 排序能力很好；
- 绝对分数仍不稳定；
- 验证阈值依然过高；
- 概率并没有完成良好跨域校准。

因此 Stage 2 虽然改善了外部召回率，但没有真正解决阈值迁移问题。

14. 这次实验的正确科学意义

这次实验应当被定义为：

> Leaky negative control / supervised target-domain adaptation upper-bound experiment

它能支持以下结论：

- Wednesday 标签对 Stage 2 有很高的可学习价值；
- source-host上下文中存在强烈的异常信息；
- Stage 2模型容量足够；
- Stage 2训练和评估管线能够正常工作；
- 获得目标域监督后，AUC可以从0.77提高到约0.95；
- Stage 1 embedding并没有丢失所有Wednesday攻击信息；
- 上下文模型能够利用源主机活动结构放大异常信号。

它不能支持以下结论：

- Stage 2在未见Wednesday数据上达到AUC 0.947；
- Stage 2比Stage 1真实提高29.9个百分点F1；
- source-host上下文本身贡献了全部增益；
- 模型具有可靠的零样本跨域泛化能力；
- 当前结果可以作为论文主结果或最终测试结果。

15. 下一步应如何设计干净实验

实验一：完全无泄漏 Stage 2

训练：

```
target rows = train only
```

验证：

```
val only    early stopping
```

测试：

```
Wednesday 5000
```

这是衡量纯跨域泛化的正式实验。

实验二：合法目标域适配

如果目的是研究有监督域适配，可以按时间划分：

```
Wednesday 2500  adaptation/calibration
```

必须满足：

- 后2500条标签不进入训练；
- 后2500条不用于early stopping；
- 后2500条不用于阈值选择；
- 最终只报告后2500条指标。

如果 source host 上下文允许在线继承前半段历史，这是合理的，但要在论文中明确说明。

实验三：区分“上下文增益”和“目标域标签增益”

至少需要以下四组：

```
| 实验 | 上下文 | Wednesday标签进入训练 |  
|---|---|---|  
| A | 无, target-only | 否 |  
| B | source-host | 否 |  
| C | 无, target-only | 是, 前50% |  
| D | source-host | 是, 前50% |
```

由此可以分别估计：

```
B - A =  
C - A =  
D - C =
```

当前只有D，无法判断大幅提升究竟来自上下文还是泄漏标签。

最终评价

这次运行在工程上是成功的，在负对照意义上也非常有价值：

> 有50%验证和50%测试目标标签参与训练后，Stage 2将Wednesday外部ROC-AUC提升到0.9471、PR-AUC提升到0.9115，并把异常召回率从22.38%提升到51.74%。

但它是明确的 leaky run：

> 2,500条测试样本和26,368条验证样本直接参与了训练，因此所有验证和测试指标均被污染，不能作为正式泛化结论。

最合理的用途是将它保留为“目标域监督上界/泄漏负对照”。下一步应该运行完全clean版本，以及“前50%适配、只评估后50%”的合法域适配版本。只有这两组结果齐全，才能判断Stage 2 source-host上下文本身带来了多少真实提升。